Vision embarquée sur FPGA

Vers une IA embarquée sûre et certifiable : état de l'art établi à partir du corpus PaperDatabase

Léonard Rivals

1er octobre 2026 — brouillon

Résumé

Ce document fait l'état de l'art de la vision par réseaux de neurones sur FPGA : accélérateurs CNN, toolflows, détection d'objets de type YOLO, autoencodeurs, apprentissage continu embarqué, et ce qu'il faudrait pour certifier un tel système. Il s'appuie sur les 101 fiches de lecture et les 216 mesures sur FPGA d'une base de 101 articles, en donnant la source de chaque chiffre et la portée de chaque puissance. Il en ressort cinq constats. Les toolflows dataflow (FINN, hls4ml, fpgaConvNet, SATAY) sont matures pour de petits réseaux, mais bornés par la mémoire on-chip dès qu'on vise un détecteur ou un autoencodeur d'images sur une petite puce. La détection sur FPGA manque d'un protocole de mesure commun : ses 87 mesures emploient 15 métriques de précision différentes, 32 seulement portent une mAP COCO, et aucune ne réunit mAP standard, débit de bout en bout et puissance de carte mesurée ; aucune ne donne de pire cas de latence. Un seul article détecte des anomalies d'images par autoencodeur sur FPGA embarquée, sur des images hyperspectrales et avec un seuil réglé sur le test ; aucun ne le fait sur des images RGB réelles. L'apprentissage sur FPGA existe, rétropropagation comprise, mais il est lent, en flottant, d'énergie mal mesurée, et aucun ne porte sur la détection. Enfin, les travaux de sûreté (fiabilité, vérification, surveillance à l'exécution, guides de certification) et les accélérateurs de détection ne se rejoignent pas : les premiers portent sur de petits classifieurs, les seconds ignorent la sûreté, et les guides renvoient l'implémentation sur FPGA à plus tard. Le document termine sur les gaps et quatre questions de thèse candidates.

Table des matières

1 Introduction **2**

2 Méthode et conventions **3**

3 Architectures d'accélérateurs CNN sur FPGA **4**

3.1 Le modèle roofline et l'exploration de l'espace de conception 4

3.2 Deux familles : moteur de calcul unique et architecture *streaming* 4

3.3 La mémoire on-chip, goulot de l'architecture streaming 4

4 Toolflows : du réseau entraîné au bitstream **5**

4.1 FINN et FINN-R : réseaux binaires puis quantifiés 5

4.2 fpgaConvNet : partitionner ou recharger les poids 6

4.3 hls4ml : la très faible latence, venue de la physique des particules 6

4.4 SATAY : un toolflow streaming pour YOLO 6

4.5 Vitis AI et la DPU : le flot par défaut, peu documenté 7

4.6 Synthèse 7

5 Détection d'objets sur FPGA **7**
5.1 Trajectoire du corpus 7
5.2 Tableau des détecteurs FPGA du corpus 8
5.3 Post-traitement et interaction PS–PL 9

6 Autoencodeurs sur FPGA **10**
6.1 Physique des particules et signaux : la très faible latence 10
6.2 Images : compression apprise et encodeurs 10
6.3 Détection d'anomalies sur petite FPGA : des signaux, pas des images 11
6.4 Bilan 11

7 Apprentissage embarqué et continu sur FPGA **12**

8 Sûreté, fiabilité et certification **13**
8.1 Cadres de certification et guides d'assurance 13
8.2 Fiabilité matérielle : SEU, radiations, injection de fautes 14
8.3 Vérification formelle 15
8.4 Surveillance à l'exécution et incertitude 15
8.5 Logiciel prédictible et traçabilité de l'implémentation 16
8.6 Détection de personnes et robustesse 16
8.7 Bilan 17

9 Analyse critique : ce que les chiffres publiés permettent de conclure **18**
9.1 La puissance : rarement mesurée sur la carte, jamais sur la puce FPGA 18
9.2 Des métriques qui flattent 18
9.3 La précision n'est presque jamais remesurée sur la cible 18
9.4 Baselines GPU embarqué faibles ou absentes 19
9.5 Reproductibilité 19

10 Zones non couvertes et pistes de recherche **19**
10.1 Quatre questions de thèse candidates 20

11 Conclusion **23**

1 Introduction

Un FPGA embarqué, par exemple un SoC Zynq ou un module Kria, associe des processeurs ARM à une logique programmable. On peut y implanter un accélérateur taillé pour un réseau donné, avec la précision numérique de son choix. Les promesses sont connues : une latence déterministe, une bonne efficacité énergétique et une précision arbitraire. Les contraintes aussi : peu de mémoire on-chip, des outils de conception lourds et une chaîne de compilation qui peut dégrader le modèle.

Le cas d'application visé est un YOLO embarqué de vidéosurveillance, qui détecte et identifie des personnes dans une zone industrielle dangereuse. Un tel système doit être rapide et sobre, mais aussi *sûr* : il faut pouvoir montrer qu'il détecte les personnes avec un taux de défaillance acceptable, et que cette preuve vaut pour le circuit qui tourne sur la carte. Ce document répond à quatre

questions, à partir du corpus seulement :

- quelles architectures et quels outils permettent aujourd'hui de porter un réseau de vision sur FPGA, et avec quelles limites (sections 3 et 4) ?
- où en sont les trois applications visées par la thèse : la détection YOLO, les autoencodeurs et l'apprentissage continu (sections 5 à 7) ?
- que sait-on garantir d'un tel système : fiabilité face aux fautes matérielles, vérification, surveillance à l'exécution, cadres de certification (section 8) ?
- que peut-on conclure des chiffres publiés, et que reste-t-il à faire (sections 9 et 10) ?

Les Jetson, MCU et NPU n'apparaissent ici que comme baselines. Une version élargie à toute la vision embarquée, qui les traite pour eux-mêmes, existe à part.

2 Méthode et conventions

Corpus. Ce document ne s'appuie que sur la base PaperDatabase. Il a été écrit en deux temps. Les sections thématiques (architectures, toolflows, détection, autoencodeurs, apprentissage continu, comparaisons et critique) décrivent la base du 26 septembre 2026 : 45 articles, 41 fiches de lecture complètes et 172 lignes de mesures dans `benchmarks.csv` (31 articles, dont 101 lignes mesurées sur FPGA) ; leurs comptes sont ceux de cette date. La section sur la sûreté et les sections de gaps reposent sur la base du 1er octobre 2026 : 101 articles, tous fichés, et 299 lignes de mesures (54 articles, dont 216 lignes sur FPGA). Les 60 articles ajoutés entre-temps (surtout sûreté et certification, détection de personnes, quelques détecteurs sur FPGA) ne sont donc repris dans les sections thématiques que lorsqu'ils contredisent un constat. Aucune référence extérieure à la base n'est citée comme source.

Origine des chiffres. Chaque valeur est reprise de la fiche ou de `benchmarks.csv`, avec la section ou le tableau de l'article entre crochets, par exemple [Table 8]. Une valeur que les auteurs ne donnent pas s'écrit `n/r` : elle n'est jamais estimée. Les chiffres d'un survey sont ceux des travaux qu'il cite. Ils ne sont pas vérifiés à la source et servent à orienter la lecture, pas à comparer. Les lignes de `benchmarks.csv` en confiance *medium* viennent de tableaux reconstruits automatiquement (Docling, PyMuPDF) ou de valeurs lues dans le texte qui commente une figure. Elles sont signalées par ^{m} dans les tableaux. Une seule ligne est en confiance *low* ; elle est signalée dans le texte.

Portée de la puissance. Une puissance n'a de sens qu'avec sa portée. La base distingue *board* (carte entière, au wattmètre), *chip* (rails de la puce ou télémétrie embarquée), *board-dynamic* (carte, repos soustrait), *estimated* (outil de conception, par exemple le rapport de Vivado) et *unspecified* (méthode non décrite). Les tableaux donnent cette portée à côté de chaque valeur. Deux puissances de portées différentes ne se comparent pas.

Lu contre déduit. Sauf mention, une affirmation résume ce que disent les auteurs. Une inférence de la fiche ou de ce document est marquée *(déduit)*. Un article marqué ^{PP} est un preprint non relu par les pairs (au moins dans la version ingérée).

Limites de la méthode. Le corpus est petit (45 articles) et vient d'une bibliographie de départ constituée à partir de résumés. Les effectifs par axe sont déséquilibrés : 24 fiches sur les plateformes, 14 sur les toolflows, 11 sur les autoencodeurs, 8 sur YOLO sur FPGA, 8 surveys et 5 sur l'apprentissage continu. Un « gap » signifie donc qu'un sujet est absent *de ce corpus*. Son absence de la littérature reste une hypothèse tant qu'une recherche ciblée ne l'a pas confirmée. Enfin, les fiches ont été rédigées à partir du texte extrait des PDF : les valeurs données uniquement dans des figures sont souvent `n/r`.

3 Architectures d'accélérateurs CNN sur FPGA

3.1 Le modèle roofline et l'exploration de l'espace de conception

Zhang et al. (2015) posent le problème qui structure la suite : à ressources égales, deux accélérateurs peuvent différer de 90 % en performance, parce que le débit de calcul ne correspond pas à la bande passante mémoire [§1]. Ils appliquent le modèle *roofline* : la performance atteignable est le minimum entre le toit de calcul et le produit du ratio calcul/communication par la bande passante [§2.3, Éq. 1]. Le tuilage des boucles de la convolution fixe ces deux grandeurs, et une énumération exhaustive (au plus 98 000 designs légaux, explorés en 10 minutes) choisit la meilleure configuration [§3.4–3.5]. Des facteurs de déroulage uniformes pour toutes les couches restent à moins de 5 % de l'optimum par couche et évitent de reprogrammer le FPGA entre couches [Table 4].

Le résultat, 61,62 GFLOPS sur les convolutions d'AlexNet en FP32 sur un Virtex-7 VX485T pour 18,61 W mesurés sur la carte [Tables 7–8], est daté : calcul flottant, et seules les convolutions sont accélérées (ni pooling, ni couches denses, [§2.1]). Le gain énergétique de 24,6× sur le CPU repose sur le TDP du Xeon et non sur une mesure [§5.2, Table 8]. La méthode, elle, reste la référence du dimensionnement d'un accélérateur.

3.2 Deux familles : moteur de calcul unique et architecture *streaming*

Les deux surveys de 2018 du corpus proposent des taxonomies complémentaires. Abdelouahab et al. (2018)[PP] classent les optimisations en trois familles [§2.4.4, Fig. 2] :

- les *transformations de calcul* : GEMM, Winograd (2,25× moins de multiplications pour $F(2\times2, 3\times3)$), FFT [§3] ;
- l'*optimisation du chemin de données* : réseaux systoliques, SIMD, déroulage et tuilage des boucles, dataflow [§4] ;
- le *calcul approché* : virgule fixe, réseaux binaires ou ternaires, élagage, rang faible [§5].

Ils rappellent que les convolutions représentent plus de 90 % du temps de calcul et les couches denses plus de 90 % des poids [§2.4.1, Table 2].

Venieris et al. (2018) recensent 13 toolflows et les séparent selon leur architecture [§2.3] :

- **streaming** (un bloc matériel par couche, en pipeline) : fpgaConvNet, DeepBurning, Haddoc2, AutoCodeGen, FINN. Ces designs sont efficaces, mais il faut un bitstream par réseau, et la taille du réseau est bornée par la mémoire on-chip ;
- **moteur de calcul unique**, piloté par logiciel (réseau systolique ou produit matriciel) : Angel-Eye, ALAMO, DnnWeaver, Caffeine, FP-DNN, Snowflake, SysArrayAccel, FFTCodeGen. Un seul bitstream sert plusieurs réseaux.

Leurs enseignements [§2.7] : le RTL optimisé bat le HLS, une exploration plus fine donne de meilleurs designs, et les moteurs uniques profitent des réseaux réguliers (VGG16) mais souffrent des noyaux hétérogènes (AlexNet). Ils proposent aussi une grille d'évaluation [§3.1] : débit en GOp/s avec le nombre d'opérations, les poids et le batch, latence à batch 1, ressources, puissance, précision applicative et répartition mémoire on-chip/off-chip. Les métriques normalisées (GOp/s par DSP ou par kLUT) ne valent, selon eux, qu'à l'intérieur d'une même famille de FPGA.

Deux réserves sur ce survey : ses chiffres sont repris des publications ou des auteurs, sans qu'aucun outil soit relancé [§2.6], et ses tableaux comparatifs ne contiennent ni puissance ni précision. Les auteurs du survey sont aussi ceux de fpgaConvNet, ce qui invite à lire avec recul les comparaisons qui le concernent *(déduit)*.

3.3 La mémoire on-chip, goulot de l'architecture streaming

Une architecture streaming garde les poids en mémoire on-chip. Sur une petite puce, c'est la BRAM qui limite. Petrica et al. (2020)[PP], de l'équipe FINN, en donnent la cause : la forme des

mémoires de poids, dictée par le parallélisme (PE × SIMD), s'accorde mal aux blocs de 18 Kb, dont une grande partie reste vide [§II-B]. Le rendement d'occupation des BRAM tombe à 52,9 % pour un ResNet-50 quantifié sur Alveo U250 [Table IV]. Leur technique, FCMP, sur-cadence la mémoire par rapport au calcul et regroupe plusieurs tampons de poids par BRAM :

— jusqu'à 30 % de mémoire on-chip en moins, pour environ 5 % de LUT en plus [§I] ;
— le classifieur CNV-W1A1 (CIFAR-10) passe du Zynq 7020 au 7012S sans perte de débit [Table V] ;
— ResNet-50 (W1A2, 67,3 % de top-1) tourne en dataflow sur une seule puce, mais sur une Alveo U250 : 2 703 FPS, 1,9 ms de latence, 71 W dont la méthode de mesure n'est pas décrite [Table II].

L'empaquetage des activations (FIFO, tampons de lignes) reste en travail futur [§VI].

D'autres réponses au même problème apparaissent dans le corpus :

— le rechargement des poids de fpgaConvNet (section 4) ;
— la compression structurée de REQ-YOLO, qui fait tenir tous les poids en BRAM (section 5) ;
— la FIFO logicielle de SATAY, qui déporte en DDR les plus gros tampons des connexions résiduelles (section 4).

Côté MCU, le même verrou existe pour la SRAM des activations : Lin et al. (2021) le contournent par une inférence par patchs, qui divise le pic mémoire par 4 à 6 sur un STM32F746 [Fig. 6]. Transposer cet ordonnancement par patchs à la BRAM d'un accélérateur streaming n'est pas traité dans le corpus *(déduit)*.

4 Toolflows : du réseau entraîné au bitstream

Un toolflow traduit un réseau décrit dans un framework d'apprentissage en une architecture matérielle, sous les contraintes de ressources d'une puce donnée (Venieris et al., 2018, §1). Le corpus couvre quatre lignées dataflow (FINN, fpgaConvNet, hls4ml, SATAY) et le flot industriel d'AMD, Vitis AI, qui exécute le réseau sur un moteur unique (la DPU).

4.1 FINN et FINN-R : réseaux binaires puis quantifiés

Umuroglu et al. (2016) partent d'un constat : sur une Zynq UltraScale+ ZU19EG, le pic binaire (66 TOPS) vaut environ 16 fois le pic 8 bits, et un AlexNet binarisé tiendrait on-chip [§3.1]. FINN génère une architecture streaming, un moteur par couche, tous les poids en mémoire on-chip [§4.1]. Trois idées de ce papier sont reprises partout ensuite :

— l'accumulation par *popcount*, qui coûte à peu près moitié moins de LUT qu'une accumulation signée [§4.2.1] ;
— le remplacement de la batch normalization suivie du signe par un simple seuil [§4.2.2] ;
— le *folding* (P PE × S voies SIMD par couche), qui équilibre les débits des couches [§4.4].

La mesure est soignée : puissance de la puce (P_{chip}, lue par PMBus) et de la carte (P_{wall}, au wattmètre), moyennées sur 10 s [§5.1]. Les réseaux sont petits : MLP sur MNIST, et un CNN (CNV) sur CIFAR-10 à 80,1 % de précision. CNV-max atteint 21 900 images/s pour 3,6 W de puce et 11,7 W de carte sur une ZC706 [Tables 3–4]. La carte consomme environ 7 W au repos [§5.2.2] : un FPS/W calculé sur la carte ne se transpose donc pas à une carte plus sobre *(déduit)*.

Blott et al. (2018) étendent FINN aux précisions multiples. FINN-R est structuré comme un compilateur, avec une représentation intermédiaire qui connaît la quantification de chaque nœud [§4]. Il apporte :

— des modèles de coût : les LUT d'un produit scalaire sont proportionnelles au produit (nombre d'éléments × bits de poids × bits d'activation), et le modèle prédit la synthèse à ±30 % près [§3.1–3.2] ;
— deux architectures : *dataflow* (un moteur par couche) et *multilayer offload* (MO), qui réutilise une partie du pipeline en boucle avec les cartes d'activation en mémoire externe, pour les

réseaux trop gros [§3.3].

Sur petite cible, le MO fonctionne mal. Tincy YOLO (W1A3) n'atteint que 133 GOp/s sur PYNQ-Z1 et 288 GOp/s sur Ultra96, avec une prédiction juste à 66–68 % seulement [Table 5]. Le papier ne donne aucun FPS pour ce détecteur. L'outil ne gère que les ressources et le débit : latence et puissance ne sont pas modélisées [§4].

4.2 fpgaConvNet : partitionner ou recharger les poids

Venieris and Bouganis (2017) modélisent le réseau en *synchronous dataflow* et explorent quatre transformations [§2.4] :

— le partitionnement avec reconfiguration du FPGA, amortie par lots, qui vise le débit ;
— le repli à gros grain (nombre d'unités par couche) ;
— le repli à grain fin (parallélisme du produit scalaire) ;
— le rechargement des poids sur une architecture unique, sans reconfiguration, qui vise la latence.

C'est la seule lignée du corpus conçue pour des réseaux qui ne tiennent pas on-chip. Elle calcule en virgule fixe 16 bits et ne descend pas plus bas ; FINN-R le relève (Blott et al., 2018, §2.2). La comparaison au Tegra X1 est l'une des rares du corpus à budget de puissance égal (5 W) et avec la même méthode de mesure des deux côtés : jusqu'à 5,53× le débit du GPU, et jusqu'à 6,65× en mode latence [§3.1]. Deux réserves : le GPU est bridé à 76,8 MHz, et les valeurs absolues ne figurent que dans une figure. Le papier, un article de workshop, ne donne ni ressources ni précision.

4.3 hls4ml : la très faible latence, venue de la physique des particules

hls4ml est né pour le déclenchement de premier niveau du LHC, avec une latence de l'ordre de la microseconde. Chaque couche y devient un bloc matériel dont le parallélisme se règle par le *reuse factor* (Fahim et al., 2021, §4.2). Le volet CNN (Aarrestad et al., 2021) traite la convolution en flux, avec une fenêtre glissante [§3]. Sur un petit classifieur SVHN (12 858 poids) :

— en quantification post-entraînement (PTQ), la précision s'effondre sous 14 bits ;
— en quantification pendant l'entraînement (QAT, avec QKeras), elle tient jusqu'à 3 à 4 bits [§7–8] ;
— le modèle QAT élagué sur 7 bits prend 174 DSP au lieu de 6 377 sur VU9P, pour une latence d'environ 5 µs [Table 3] ;
— sur XC7Z020, il atteint 2 831 images/s pour 171 µs de latence [Table 4].

Aucune puissance FPGA n'est mesurée : l'énergie citée est une estimation ASIC 45 nm.

La version 2025 (Schulte et al., 2025)^{PP} ajoute l'arithmétique distribuée sans DSP et la quantification par paramètre (HGQ). Elle s'évalue en placement-routage hors contexte, sur de petits modèles, sans puissance, et ses auteurs reconnaissent que les gros modèles échouent en synthèse [§9, §11]. Campos et al. (2024) relient la quantification mixte guidée par la Hessienne (HAWQ, largeurs choisies par programmation linéaire en nombres entiers sous budget de BOPs) à hls4ml via QONNX. Sur un MLP, ils obtiennent −95,7 % de DSP pour −1,07 point de précision [Table 2]. Mais les latences sont estimées par Vivado HLS, aucun design ne tourne sur carte [§6.2], et la puce cible est ambiguë (voir la fiche). Enfin, Govorkova et al. (2026) ajoutent un backend pour les FPGA flash durcis Microchip PolarFire, limité aux couches denses et ReLU [§5.1].

4.4 SATAY : un toolflow streaming pour YOLO

Montgomerie-Corcoran et al. (2023) sont les premiers du corpus à automatiser une architecture streaming pour la détection (YOLOv3-tiny, YOLOv5, YOLOv8), de l'ONNX au bitstream [§IV]. Leurs réponses aux opérations propres à YOLO :

— HardSwish en remplacement de SiLU [§III-B] ;

— redimensionnement (*resize*) calculé à la volée [§III-B] ;
— FIFO logicielle en DDR pour les longues connexions résiduelles. Déporter les 5 plus gros tampons réduit la mémoire on-chip totale de 17 % [§V, Fig. 9].

Tous les poids restent on-chip (jusqu'à 26 MB, [§IV-C]). La seule carte embarquée testée est la ZCU104, avec YOLOv5n (section 5).

4.5 Vitis AI et la DPU : le flot par défaut, peu documenté

Dans le corpus, Vitis AI n'apparaît que comme flot utilisateur : YOLOv5 INT8 sur Kria KV260 (Al Amin and Obermaisser, 2025) et RetinaNet ResNet-50 INT8 sur ZCU104 et KV260 (Magalhães et al., 2022). Ce dernier papier montre le coût d'adaptation du modèle : il faut dupliquer une convolution du FPN pour que tout le réseau tienne sur la DPU sans repli sur le CPU [§2.3]. Vitis AI n'a que 2 lignes dans `benchmarks.csv`, alors que c'est le flot par défaut du KV260.

4.6 Synthèse

Table 1 – Toolflows du corpus (lu dans les fiches). « Puissance » : existe-t-il une mesure sur carte dans les articles du corpus ?

Toolflow	Architecture	Précisions	Tâches démontrées	Puissance mesurée
FINN / FINN-R	streaming ; MO pour gros réseaux	binaire à 8 bits	classification ; Tincy YOLO en MO	oui (puce et carte, FINN ; carte, FINN-R)
fpgaConvNet	SDF ; partitionnement ou rechargement des poids	FXP16	classification (AlexNet à DenseNet)	dynamique de carte, repos soustrait
hls4ml	un bloc par couche, reuse factor	virgule fixe arbitraire, HGQ	MLP, petits CNN, AE et VAE (physique)	non (estimations seulement)
SATAY	streaming, FIFO logicielle DDR	W8A16 (PTQ)	YOLOv3-tiny à v8	méthode non décrite
Vitis AI (DPU)	moteur unique	INT8	YOLOv5, RetinaNet	méthode non décrite ou valeurs en figure

Aucun article du corpus ne compare deux de ces flots, mesurés, sur un même réseau et une même carte. Les papiers d'outil comparent des chiffres publiés (Venieris et al., 2018, §2.6) ou ne comparent pas (Schulte et al., 2025; Fahim et al., 2021). QONNX, lisible par FINN et par hls4ml, rend cette comparaison techniquement faisable (Campos et al., 2024, §2.3).

5 Détection d'objets sur FPGA

5.1 Trajectoire du corpus

Les premiers travaux sur la détection partent d'un constat de Ding et al. (2019, §1) : la littérature FPGA accélère surtout la classification. En 2018, aucun toolflow ne visait la détection (Venieris et al., 2018, §2.6, §3.1). Le corpus montre ensuite quatre approches.

Compression puis design sur mesure. REQ-YOLO compresse Tiny YOLO (YOLOv2-tiny) de deux façons :

— des matrices bloc-circulantes, qui transforment la convolution en FFT, produit terme à terme et FFT inverse [§3] ;
— une quantification hétérogène par ADMM qui choisit, couche par couche, entre des multiplications sur DSP et des décalages sur LUT [§4].

Tous les poids tiennent alors en BRAM [§5]. Sur un Virtex-7 690T en carte PCIe, le modèle atteint 314,2 FPS pour 21 W [Table 2]. Trois réserves :

— la latence de 3,18 ms est une somme des latences par couche, sans transfert PCIe ni NMS [Table 1] ;
— la mAP absolue n'est pas dans le texte : seules des pertes relatives le sont [§6.2] ;
— la plateforme n'a rien d'embarqué.

Face à une Jetson TX2 exécutant le même modèle (28,4 FPS, 10,8 W, [Table 2]), le gain est net, mais TensorRT n'est pas mentionné : la TX2 est vraisemblablement sous-exploitée *(déduit)*.

Protocole commun, mesuré par un tiers. Le concours DAC-SDC 2018 (Xu et al., 2021) impose deux plateformes (PYNQ-Z1 et Jetson TX2), un jeu de test caché d'images de drone, un débit minimal et une énergie mesurée au wattmètre par les organisateurs [§2.3]. C'est le seul protocole commun FPGA/GPU du corpus. Les équipes FPGA gagnantes utilisent des détecteurs maison en HLS ou RTL sur XC7Z020, avec les cartes d'activation en DDR [§4.2] :

— TGIIF, un SSD élagué en 8 bits : IoU de 0,62 à 11,96 FPS [§5.2.1] ;
— SystemsETHZ, un SqueezeNet binaire suivi d'une tête YOLO : IoU de 0,49 à 25,97 FPS [§5.2.1].

Limites : la métrique est l'IoU d'un seul objet par image, pas une mAP ; les puissances par équipe ne figurent qu'en figures ; le score composite utilise des bases de logarithme différentes pour le FPGA et le GPU [§2.3]. Les auteurs estiment que le FPGA consomme 1/3 à 1/2 de l'énergie du GPU pour une précision proche, sans chiffre dans le texte [§5.3].

Design HLS sur SoC d'entrée de gamme. Zhai et al. (2023) élaguent YOLOv3-tiny par canaux (calcul divisé par 7,4), le quantifient en virgule fixe 16 bits dynamique et l'implantent en HLS sur un Zynq XC7Z035. Leur accélérateur combine Winograd pour les convolutions 3×3 et im2col + GEMM pour les 1×1 [§3.1, §3.5]. Le NMS et le suivi Deepsort tournent sur l'ARM [§3.3]. Avec un module, ils obtiennent 91,65 FPS pour 12,51 W ; avec deux modules, 168,72 FPS pour 15,18 W [Tables 8–9]. La méthode de mesure de puissance n'est pas décrite. Deux points critiques :

— l'AP@0.5 est identique en FP32 et en INT16 à trois décimales, ce qui laisse penser qu'elle n'a pas été remesurée sur la cible *(déduit)* ;
— la ligne Jetson Nano de leur comparaison vient d'un autre article et porte sur un modèle non élagué [Table 7]. Le gain affiché vient donc surtout de l'élagage.

Toolflow streaming et flot industriel. SATAY (section 4) couvre YOLOv3-tiny à YOLOv8s, mais ses meilleurs chiffres viennent de cartes Virtex de 23 à 67 W [Table III]. Sur la seule carte SoC testée, la ZCU104, YOLOv5n en 640×640 obtient 26,7 de mAP50 COCO pour 21,41 ms et 14,5 W, contre 27,7 de mAP50, 32,28 ms et 8,58 W pour une Jetson TX2 [Table IV]^m. L'énergie par inférence est du même ordre (317 contre 277 mJ, [Table IV]). À l'autre extrémité, Al Amin and Obermaisser (2025) déploient YOLOv5 INT8 par Vitis AI sur KV260 : 9 FPS et 3,5 W, méthode non décrite [Table I]. Leur précision (98,6 %) est une « classification accuracy » sur 72 images choisies, qui ne se compare à aucune mAP. Leur propre tableau contredit leur conclusion : 2,6 FPS/W, sous quatre des cinq travaux cités [§IV-B, Table I].

5.2 Tableau des détecteurs FPGA du corpus

Sources : REQ-YOLO (Ding et al., 2019, Tables 1–2) ; FINN-R (Blott et al., 2018, Table 5) ; DAC-SDC (Xu et al., 2021, §5.2.1) ; Zhai (Zhai et al., 2023, Tables 8–9) ; SATAY (Montgomerie-

TABLE 2 – Détecteurs sur FPGA dans `benchmarks.csv`. Précisions non comparables d'une ligne à l'autre (jeux, métriques et résolutions différents). ^{m} : confiance *medium*. Portée de puissance : u = *unspecified*, b = *board*.

Travail	Modèle, précision	Puce	Métrique	Préc.	FPS	ms	W (portée)
REQ-YOLO	Tiny YOLO bloc-circulant, bits `n/r`	XC7VX690T	mAP VOC	`n/r`	314,2	3,18	21 (u)
FINN-R	Tincy YOLO W1A3 (MO)	XC7Z020	mAP VOC	50,1	`n/r`	`n/r`	2,5 (b)
FINN-R	Tincy YOLO W1A3 (MO)	XCZU3EG	mAP VOC	50,1	`n/r`	`n/r`	9,7 (b)
DAC-SDC	TGIIF : SSD élagué INT8	XC7Z020	IoU, 1 objet	62,4	11,96	`n/r`	`n/r`^{m}
DAC-SDC	SystemsETHZ : SqueezeNet binaire + YOLO	XC7Z020	IoU, 1 objet	49,2	25,97	`n/r`	`n/r`^{m}
Zhai	YOLOv3-tiny élagué INT16, 1 module	XC7Z035	AP@0.5, 1 classe	59,9	91,65	`n/r`	12,51 (u)
Zhai	idem, 2 modules	XC7Z035	—	`n/r`	168,72	`n/r`	15,18 (u)
SATAY	YOLOv5n 640, PTQ	ZCU104	mAP50 COCO	26,7	`n/r`	21,41	14,5 (u)^{m}
SATAY	YOLOv8s 640, W8A16	VCU118	mAP* COCO	61	`n/r`	24,5	57,4 (u)^{m}
Magalhães	RetinaNet R50 INT8, 2 DPU	ZCU104	mAP VineSet	`n/r`	25	`n/r`	`n/r`^{m}
Al Amin	YOLOv5 INT8 (DPU)	KV260	acc., 72 images	98,6	9	`n/r`	3,5 (u)

Corcoran et al., 2023, Tables III–IV) ; Magalhães (Magalhães et al., 2022, §3) ; Al Amin (Al Amin and Obermaisser, 2025, Table I). La latence de REQ-YOLO est une somme des latences par couche. Le mAP* de SATAY porte un astérisque dont la définition n'a pas été extraite.

Lecture du tableau. D'après un recompte de `benchmarks.csv`, sur 29 lignes de détection sur FPGA, 14 ont une mAP COCO, et toutes viennent de SATAY. Aucune ligne ne rapporte à la fois une mAP standard, un FPS de bout en bout et une puissance de carte mesurée. Les surveys de 2025 confirment ce manque d'homogénéité :

— Hozhabr and Giorgi (2025) proposent de normaliser par le *pixel throughput* (FPS × pixels par image, [§II-C2]). Cela corrige la résolution, mais ni la précision ni la portée de puissance ;
— Sali et al. (2025)^{PP} alignent des designs mesurés dans des conditions hétérogènes, avec des incohérences entre texte et tableaux (voir la fiche).

Hozhabr et Giorgi affirment que « FPGAs have emerged as the optimal hardware platform, offering both speed and power efficiency » [§IV], sans comparaison mesurée avec une Jetson récente ni un MCU à NPU.

5.3 Post-traitement et interaction PS–PL

Le NMS, le suivi et le redimensionnement tournent presque toujours sur le processeur du SoC et sont rarement comptés :

— Zhai et al. : NMS et Deepsort sur l'ARM, inclus ou non dans le FPS, on ne sait pas [§3.3] ;
— REQ-YOLO : latence sans NMS [Table 1] ;
— Magalhães et al. : cœur du réseau seul, pré- et post-traitement exclus [§2.5] ;
— DAC-SDC : redimensionnement sur l'ARM [§4.2].

Le survey de Sali et al. cite un cas où le débit passe de 195 FPS pour la détection seule à 24 FPS de bout en bout, à cause d'un post-traitement en Python [§VI-C] (chiffre d'un travail cité, non vérifié). Hozhabr et Giorgi citent deux contre-exemples, avec un NMS pipeliné en matériel ou un système entièrement sur FPGA [§V-A]. Tous deux sont sur de grosses puces (Stratix 10, Kintex-7) et restent à lire à la source.

6 Autoencodeurs sur FPGA

La bibliographie de départ supposait que les autoencodeurs (AE) sur FPGA étaient surtout étudiés sur des séries temporelles et en physique des particules, et peu sur des images. Les 11 fiches de l'axe permettent de préciser ce constat.

6.1 Physique des particules et signaux : la très faible latence

Govorkova et al. (2021) placent des AE et VAE au déclenchement de premier niveau du LHC, pour sélectionner des collisions anormales sans hypothèse sur la nouvelle physique. Trois choix sont transposables à d'autres domaines :

— le score d'anomalie (erreur quadratique) est calculé sur la puce, par une couche ajoutée à hls4ml [§IV] ;
— le VAE est réduit à son encodeur : le score s'obtient à partir de la divergence de Kullback-Leibler (D_{KL}) calculée sur (μ, σ), ce qui supprime l'échantillonnage et le décodeur [§IV] ;
— le choix entre QAT et PTQ se fait selon la stabilité du score d'anomalie, et non selon la perte d'entraînement [§VI].

Le VAE dense (encodeur seul) tient en 80 ns sur VU9P ; le seul AE convolutif, sur une entrée de 19×3 valeurs, prend 47 % des LUT et est rejeté [Table III]. Ces chiffres sont des estimations de Vivado HLS ; une seule validation sur carte est faite, et elle est qualitative [§VII].

Deux autres travaux restent sur des signaux :

— Govorkova et al. (2026) compressent des impulsions de calorimètre (32 échantillons en 2 valeurs) sur un PolarFire durci : 25 ns de latence en version « nano », sans bloc mathématique [§5.2]^m. La puissance est estimée par l'outil ;
— Que et al. (2024) sont les seuls à échantillonner en matériel (générateur Mersenne Twister suivi d'une transformée de Box-Muller), avec un latent de dimension 1, sur des séries d'ondes gravitationnelles [§III]. Une inférence dure 17 cycles (85 ns) ; les « 5,6 ns » mis en avant sont un débit inverse amorti sur 10 000 inférences [§IV-E]. La puissance est estimée par Vivado [§IV-E].

Enfin, Campos et al. (2024) quantifient un encodeur de compression de données de calorimètre (ECON-AE), dont seul l'encodeur est synthétisé [§7, Table 3].

6.2 Images : compression apprise et encodeurs

Compression apprise sur grosse FPGA. F-LIC (Sun et al., 2022) implante un codec d'images appris (VAE avec hyper-prior) en virgule fixe 8 bits sur VCU118. Il encode du 720p à 40,69 FPS et du 1080p à 19,15 FPS, avec un PSNR de 29,94 dB à 0,2542 bpp [Fig. 6]^m. Deux techniques sont réutilisables pour un décodeur :

— deux poids partagent un même DSP (« casDSP ») ;
— la déconvolution saute les zéros insérés par le stride.

Deux limites : le codage entropique tourne sur le PC hôte [Fig. 1], et l'encodeur 720p occupe 4 146 DSP. L'énergie par image est comparable à celle d'une Jetson Xavier NX (1,41 contre 1,43 J en encodage 720p), pas meilleure, et la méthode de mesure n'est pas décrite [Fig. 6].

Compression à bord, sur VPU. La mission Φ-Sat-2 (Guerrisi et al., 2023) compresse les images par un AE convolutif d'un facteur 8 fixe, sans codage entropique [§II-A]. L'encodeur tourne à bord sur une VPU Myriad 2, pas sur un FPGA : 10,92 s par acquisition pour 1,4 W, contre 0,95 s sur un GPU GTX 1650 [Table V]^m. Le passage à la VPU ne coûte presque rien en qualité : SSIM de 0,9647 contre 0,9651 [§III-B–C].

Encodeurs d'images sur petite FPGA. Manjunath et al. (2021) ternarisent l'encodeur convolutif d'un agent d'apprentissage par renforcement (entrée 120×120 RGB, simulateur) et l'implantent en Verilog sur Artix-7. En 32 bits et 64 PE, une image coûte 0,8 ms et 0,25 mJ [Table 5]. La puissance est estimée par Vivado [§VIII-A], et le décodeur n'est pas implanté. Isik et al. (2023)[PP] débruitent MNIST sur XCZU7EV (2,91 ms, 5,93 W « on-chip »). C'est la seule ligne de `benchmarks.csv` en confiance *low* : l'« accuracy » d'un débruiteur n'est pas définie, et le tableau comparatif est physiquement incohérent [Table II].

6.3 Détection d'anomalies sur petite FPGA : des signaux, pas des images

— Malviya et al. (2022) détectent des anomalies de vibrations d'hélicoptère, encodées en scalogrammes 64×64, avec un AE convolutif de 8 755 paramètres sur PYNQ-Z2. Encodeur et décodeur sont dans deux bitstreams séparés (40 403 et 38 169 LUT), pour 2,1 + 3,8 ms [Table 4, §IV-B][m]. La précision est remesurée sur le FPGA et baisse (85,02 % contre 88,8 % sur CPU) en virgule fixe 23 bits [§IV-B].
— Tsukada et al. (2020) (ONLAD) apprennent en ligne un AE à une couche cachée par OS-ELM (voir section 7). Leurs données sont tabulaires ou des images aplaties.
— Coutinho et al. (2019) : malgré le titre, l'« autoencodeur empilé » n'est qu'un classifieur MLP sur MNIST, sans reconstruction [§III-A, §V-D]. Leur comparaison met une puissance dynamique de quelques mW face à des puissances totales [Tables 16–17].

6.4 Bilan

Table 3 – Autoencodeurs du corpus : donnée traitée, partie implantée et type de mesure.

Travail	Donnée	Partie en matériel	Puce	Puissance
AE du LHC [a]	tabulaire 19×3	AE complet ; VAE encodeur seul	VU9P	non mesurée
PicoCal [b]	impulsions 1D	encodeur	PolarFire	estimée
VAE de Que [c]	séries temporelles	VAE complet (latent 1)	U250, Z7045	estimée
ECON-AE [d]	calorimètre	encodeur	`n/r` (voir fiche)	non mesurée
F-LIC [e]	images 720p et 1080p	VAE hyper-prior, sans codage entropique	VCU118	méthode non décrite
Φ-Sat-2 [f]	images satellite	encodeur	VPU (pas de FPGA)	méthode non décrite
Manjunath [g]	images 120×120 (simulateur)	encodeur	Artix-7	estimée
CAE débruiteur [h]	MNIST bruité	AE complet	XCZU7EV	méthode non décrite
Malviya [i]	vibrations en scalogrammes	AE, 2 bitstreams	XC7Z020	méthode non décrite
ONLAD [j]	tabulaire, MNIST aplati	AE OS-ELM + apprentissage	XC7Z020	carte (3,1 W)
SSAE [k]	MNIST	MLP (encodeurs empilés)	XC6VLX240T	dynamique

[a] Govorkova et al. (2021) ; [b] Govorkova et al. (2026) ; [c] Que et al. (2024) ; [d] Campos et al. (2024) ; [e] Sun et al. (2022) ; [f] Guerrisi et al. (2023) ; [g] Manjunath et al. (2021) ; [h] Isik et al. (2023)[PP] ; [i] Malviya et al. (2022) ; [j] Tsukada et al. (2020) ; [k] Coutinho et al. (2019).

Trois constats se dégagent du tableau 3 :
— **l'image est traitée**, mais par la compression sur grosse FPGA (F-LIC), sur VPU (Φ-Sat-2), ou à petite échelle sur petite FPGA (Manjunath, arxiv-2301.07050[PP]) ;
— **aucun article ne détecte d'anomalies visuelles par AE sur FPGA** : les AE d'anomalie portent sur des signaux ou des données tabulaires ;

— **l'AE complet sur une petite puce reste coûteux** : au 26/09, on implantait l'encodeur seul, ou l'on coupait le réseau en deux bitstreams. Depuis, deux articles implantent encodeur et décodeur sur une UltraScale+ (CAE d'artefacts hyperspectraux sur la DPU d'une ZCU104, Castelino et al., 2024 ; compression apprise sur ZCU102, Mazouz et al., 2025). Le coût des déconvolutions sur une puce de la taille d'une KV260 reste non mesuré.

Le survey d'anomalie visuelle industrielle de Cui et al. (2023), pas encore fiché, dira quels modèles d'anomalie visuelle seraient les candidats à porter.

7 Apprentissage embarqué et continu sur FPGA

Un modèle figé ne s'adapte pas à la dérive des données. L'apprentissage sur l'appareil y répond, mais la rétropropagation coûte cher en mémoire et en énergie. Les deux seuls apprentissages sur FPGA du corpus l'évitent.

ONLAD : apprentissage séquentiel sans rétropropagation. Tsukada et al. (2020) entraînent en ligne un AE à une couche cachée par OS-ELM :

— les poids d'entrée sont aléatoires et figés, et seuls les poids de sortie sont mis à jour par moindres carrés récursifs [§2.2, §3.3] ;
— à batch 1, la seule inversion matricielle devient une division scalaire, sans changer la solution [§3.1] ;
— un facteur d'oubli léger gère la dérive : l'AUC passe de 0,55–0,59 sans oubli à 0,78–0,90 sous dérive simulée [Table 5].

Le cœur (Vivado HLS, virgule fixe 32 bits, 100 MHz) tourne sur PYNQ-Z1. C'est l'une des rares mesures de carte FPGA du corpus : 3,1 W au wattmètre, ARM compris [§6.4]. Limites : les AUC sont évaluées en TensorFlow et non sur le cœur en virgule fixe [§5–6] ; la dérive est simulée par permutation de classes ; les baselines CPU et GPU, en TensorFlow à batch 1, sont dominées par les surcoûts d'appel, ce que les auteurs reconnaissent [§6.2.1].

ImageHD : calcul hyperdimensionnel sur un extracteur figé. Arockiaraj et al. (2026) combinent un MobileNetV2 INT8 figé, un encodage en hypervecteurs binaires et une mémoire de clusters bornée, fusionnée par un kMeans++ simplifié [§IV]. L'accélérateur streaming (Vitis HLS) tient sur une ZCU104 à 300 MHz : 0,27 ms et 0,97 mJ par image sur CIFAR-10 [Tables VI–VII]. Deux réserves fortes :

— la précision de clustering est très basse (0,27 sur CIFAR-10, 0,19 sur CORe50), alors que l'article parle d'une « strong accuracy » [Table VI, §IX] ;
— latence et puissance du FPGA viennent de l'analyseur de Vitis, c'est-à-dire d'une estimation, alors que CPU et GPU sont mesurés (powertop, nvidia-smi). Les gains de 258× à 383× en énergie mélangent estimation et mesure [§VII-D].

Références hors FPGA. Deux travaux fixent des ordres de grandeur qu'un apprentissage continu sur FPGA devrait battre ou justifier :

— QLR-CL sur VEGA, un ASIC PULP 22 nm : 62 mW de puce (Ravaglia et al., 2021, §V-D) ;
— O-FSCIL sur le MCU GAP9 : 11 à 23 mJ par nouvelle classe (Wibowo et al., 2024, Table IV).

Tous deux sont détaillés dans la version complète de l'état de l'art.La seule rétropropagation embarquée du corpus (la queue d'un MobileNet-V1, en FP32) tourne sur VEGA, pas sur un FPGA (Ravaglia et al., 2021, §IV-B). Venieris et al. jugeaient déjà l'entraînement sur FPGA « à peine exploré » en 2018 (Venieris et al., 2018, §3.2).

Ce qui manque. Toutes les fiches d'apprentissage continu font de la classification ou du clustering ; `benchmarks.csv` n'a aucune ligne qui croise détection et apprentissage continu. Deux questions

restent ouvertes dans le corpus :

— une rétropropagation de la queue d'un réseau sur SoC FPGA, avec l'étage figé en logique programmable ;
— l'incrémental de classes en détection.

8 Sûreté, fiabilité et certification

Les sections précédentes jugent un accélérateur à son débit, à sa puissance et à sa précision. Pour un YOLO qui surveille des personnes dans une zone industrielle dangereuse, la question change : il ne suffit plus de dire combien d'images par seconde le système traite, il faut dire quelle preuve on peut fournir qu'il détecte les personnes avec un taux de défaillance acceptable, et que cette preuve vaut pour le réseau quantifié et compilé qui tourne sur la carte. « Sécuritaire » s'entend ici d'abord au sens de la sûreté de fonctionnement (*safety*) ; la sécurité face aux attaques (*security*) est un volet secondaire. Le corpus compte 29 fiches sur cet axe. Trois seulement mesurent quelque chose sur un FPGA, et aucune ne porte sur un détecteur d'objets.

8.1 Cadres de certification et guides d'assurance

L'EASA : learning assurance et cycle en W. Le rapport CoDANN (EASA and Daedalean, 2020) pose le vocabulaire repris ensuite : un cycle de vie en W qui ajoute au cycle en V la gestion des données, de l'apprentissage et la vérification du modèle d'inférence [§6.1], la séparation entre fonction et moniteur (*Doer/Checker*) avec un discriminateur hors distribution [§6.6], et l'allocation d'objectifs quantitatifs de 10^{-7} et 10^{-9} par heure de vol [§9.2.5, Fig. 9.3]. Deux limites comptent pour la thèse. Les bornes de généralisation théoriques sont jugées inutilisables pour de gros réseaux, si bien que les garanties reposent sur un échantillonnage i.i.d. du domaine opérationnel, lui-même difficile à démontrer [§5.3.5, §6.2.8]. Le matériel est hors périmètre : la conformité DO-254 des GPU, ASIC et FPGA « will not be addressed » [§4.2.3], et la vérification de l'étape d'implémentation est laissée à un travail futur [§6.5].

CoDANN II (EASA and Daedalean, 2024) traite justement le passage du modèle entraîné au modèle déployé. Il distingue équivalence ponctuelle, proximité ponctuelle et équivalence en moyenne entre les deux [§3.6.1], et propose d'évaluer la borne de généralisation directement sur le modèle transformé (quantifié, élagué) dans l'environnement d'inférence [§3.7.3], avec qualification des outils selon DO-178C §12.2 [§3.7.1]. Il exige que le moniteur hors distribution reçoive le même niveau d'assurance que la fonction principale [§5.2.2]. Il argumente enfin pour le FPGA contre le GPU : logique personnalisée ou Soft IP plus visibles, alors que le GPU repose sur des pilotes, un microcode et un ordonnancement propriétaires [§3.2.2, Annexe B]. Cet argument est une position d'entreprise (Daedalean), non étayée par des mesures ; les sections sur la sûreté du système et les DAL sont retirées de la version publique.

Le concept paper Issue 02 (EASA, 2024) est le document d'autorité le plus concret du corpus, mais il reste non contraignant (moyens de conformité « anticipated »). Quatre points touchent directement le sujet :

— l'apprentissage en ligne n'est pas accepté : le modèle est gelé à l'approbation [§C.1, MOC LM-02] ;
— les FPGA et ASIC relèvent des guides existants (AMC 20-152A), alors que les GPU et autres accélérateurs d'IA demanderont des guides propres [MOC IMP-05] ;
— la quantification qui change le comportement doit être traitée dans l'apprentissage [LM-06] ; en implémentation, seules les optimisations sans effet sur le comportement sont admises [MOC IMP-04-2], et la préservation des propriétés après conversion doit être vérifiée, méthodes formelles « expected to be necessary » [IMP-06] ;
— le système surveille que ses entrées restent dans l'ODD et en distribution, et que ses sorties respectent des bornes de performance et de confiance [EXP-04 à EXP-07], avec repli sur un

système classique [SRM-02].
Le seul cas d'usage en vision (Visual Landing System) tourne sur un GPU COTS non certifié [§F.2.1.3.3], et le document ne dit rien de l'équivalence numérique entre modèle entraîné et bitstream, ni des SEU.

Guides académiques. AMLAS (Hawkins et al., 2021)[PP] propose six étapes (périmètre, exigences, données, apprentissage, vérification, déploiement), chacune close par un patron d'argument en GSN. Ses exemples sont proches du cas d'application, comme l'exigence « identifier toute personne présente avec une précision d'au moins 0,93 » [Stage 2, Ex. 7]. Le matériel n'y apparaît qu'à l'intégration, pour vérifier que la plateforme cible convient au modèle [Stage 6, Ex. 45]. Ashmore et al. (2019)[PP] structurent le cycle de vie en quatre étapes et listent les desiderata de chacune ; ils incluent la surveillance des internes du modèle contre les SEU [§7.2.2] et le test sur la cible, parce que la représentation numérique change entre étapes [§7.4.1]. Le livre blanc du projet DEEL (Delseny et al., 2021)[PP] formule sept défis (évaluation probabiliste, résilience, spécifiabilité, données, explicabilité, robustesse, vérifiabilité) [§4]. Deux idées y sont directement transposables : une perte asymétrique où la non-détection d'un piéton est catastrophique et la fausse détection seulement « major » [§4.2.4.2], et la conversion d'un taux d'erreur par image en taux de défaillance par heure [§4.2.4.4]. L'implémentation est hors périmètre [§1.3]. Les surveys de Schwalbe and Schels (2020), Salay et al. (2017)[PP] et Huang et al. (2018)[PP] partent tous d'ISO 26262 ou de l'automobile. Le deuxième note qu'une fonction de perception comme « reconnaître un piéton » est peut-être inspécifiable, et recommande pour les ASIL élevés des reconnaisseurs redondants entraînés sur des modèles et des données différents [§III-C].

Un processus appliqué à un YOLO de surveillance. Belcaid et al. (2024) appliquent en partie le processus ARP6983/ED-324, encore en rédaction, à la détection de drones intrus par YOLOv3-tiny sur Jetson AGX Xavier. Ils dérivent un ODD du constituant ML (taille des objets, position, luminosité) [§II], corrigent le biais de position du jeu de données [§III], découpent l'image 1920×1080 en tuiles [Table I] et réécrivent le GEMM en CUDA pour éviter cuBLAS, fermé et difficile à analyser [§V]. Les exigences citées sont une détection supérieure à 80 % et une latence inférieure à 50 ms [§I-A] ; la latence obtenue est de 15 ms en FP16, sans dire si elle porte sur une tuile ou sur l'image [§V-B]. Aucune métrique de détection n'est publiée (confidentialité, §I-A), et la traçabilité s'arrête aux pilotes NVIDIA [§VI]. C'est le travail du corpus le plus proche du cas d'application, mais sans FPGA.

Normes industrielles : citées, pas traitées. Aucune fiche ne traite l'IEC 61508, l'ISO 13849, l'IEC 62061 ou l'ISO/IEC TR 5469 pour elles-mêmes. L'IEC 61508 n'apparaît que comme contexte (Li et al., 2017, §1, §2.3), avec la cible de moins de 10 FIT tirée d'ISO 26262, reprise pour l'ASIL D par Rech (2024, §III-C). La revue de Halme et al. (2018) s'appuie sur l'ISO 10218-1/2 et l'ISO/TS 15066 [§2] ; sur 15 méthodes de vision pour la collaboration humain-robot, seul SafetyEYE est commercial et certifié [Table 1]. Les correspondances entre les objectifs EASA ou AMLAS et les niveaux SIL ou PL ne figurent que dans les gaps rédigés par les fiches *(déduit)*.

8.2 Fiabilité matérielle : SEU, radiations, injection de fautes

Li et al. (2017) caractérisent par simulation (pas de RTL, pas de faisceau) la propagation de bit-flips transitoires dans un accélérateur de DNN. Le taux de corruptions silencieuses dépend fortement du type de données : 7,19 % en virgule fixe 32b_rb10 contre 0,38 % en FLOAT pour AlexNet [§5.1.1], et seuls les bits de poids fort comptent [§5.1.2]. Un détecteur fondé sur des bornes d'activation par couche (SED) atteint 90,21 % de précision et 92,5 % de rappel [§6.2] (l'introduction donne d'autres chiffres, §1) et ramène le FIT d'Eyeriss de 8,55 à 0,35 en FLOAT [§6.2]. Il est aveugle aux corruptions qui restent dans la plage apprise.

Les deux articles de Libano et al. sont les seuls du corpus à étudier les SEU de la mémoire de configuration d'un FPGA SRAM. Libano et al. (2019) isolent chaque couche dans un Pblock, injectent des fautes dans la mémoire de configuration et valident sous neutrons (LANSCE, ChipIR) [§III]. Sur un ANN de 19 poids (XC7Z020), le TMR de la seule couche la plus vulnérable réduit la probabilité d'erreur de 64,2 %, contre 85,1 % pour le TMR complet [§V-C, Fig. 7]. Sur un CNN MNIST de 1 848 poids (ZU9EG), il masque environ 40 % des fautes pour 8 % de ressources en plus ; le TMR complet ne tient pas dans la puce [§V-C]. Les BRAM, où résident souvent les poids, sont exclues de l'injection [§III-A]. Libano et al. (2020) binarisent les couches convolutives du même type de réseau : la section efficace estimée baisse de 39 %, mais la part d'erreurs critiques passe de 26 à 38 % [§IV-B]. Leur indicateur MEBF, qui combine inexactitude du modèle et radiations, montre qu'au niveau de la mer l'inexactitude domine [§IV-B, Fig. 5] ; les auteurs notent eux-mêmes que ce résultat, obtenu pour une inférence de 233 ns, ne se transpose pas tel quel à un détecteur [§V].

Gambardella et al. (2019) injectent des fautes « stuck-at » par canal dans des QNN FINN sur ZCU104. Sur CNV (CIFAR-10), un seul canal bloqué coûte jusqu'à environ 10,8 points de précision [Table II]. Répliquer seulement les canaux critiques coûte 49,87 % de MAC en plus pour une chute tolérée de 2 %, et 173,47 % pour 0,5 %, contre 200 % pour le TMR complet [Table III]. Les SEU de configuration sont renvoyés à un travail futur [§VII]. FT-ClipAct (Hoang et al., 2019)[PP] remplace ReLU par une activation écrêtée à un seuil appris : à un taux de fautes de 5×10^{-7} dans les poids, AlexNet garde 69,36 % de précision contre 51,16 % sans protection [§V-B, Fig. 7a], en simulation PyTorch seulement.

Deux surveys cadrent ces travaux. Ruospo et al. (2023) classent l'évaluation en quatre familles (simulation logicielle, simulation matérielle, plateforme, radiation) [Table 1] ; la simulation RTL d'un petit CNN demande environ 25 min par inférence [Discussion], et les résultats d'injection dans la mémoire de configuration varient d'un composant à l'autre et avec la température [Open challenges]. Rech (2024) rassemble les résultats sous faisceau, surtout sur GPU. Il définit une erreur critique en détection par un seuil de Jaccard sur les boîtes [§II-C] et recense des mitigations peu coûteuses : ABFT jusqu'à 85 % des erreurs critiques détectées, cohérence temporelle entre images environ 70 % [§VI-A], RP-DWC 75 % pour moins de 20 % de surcoût [§VI-B]. Ces chiffres viennent d'autres travaux et ne sont pas comparables entre eux.

Toutes les mesures portent sur des classifieurs, de 19 poids à VGG-16. Hormis le seuil de Jaccard de Rech, la criticité d'une erreur de détecteur (personne manquée contre fausse alarme) n'est pas définie, et le passage d'un FIT mesuré à un objectif SIL ou PL n'est traité nulle part *(déduit)*.

8.3 Vérification formelle

Reluplex (Katz et al., 2017)[PP] étend le simplexe aux contraintes ReLU. Sur les 45 réseaux ACAS Xu de 300 ReLU, qui remplacent une table de plus de 2 Go par moins de 3 Mo [§5], la propriété φ_1 est prouvée sur 41 réseaux en 394 517 s au total, avec 4 timeouts [Table 2], et φ_8 révèle un contre-exemple. Le solveur travaille en flottant et ne garantit pas la correction [§4] : il vérifie le modèle idéalisé, pas le réseau déployé *(déduit)*. Narodytska et al. (2017)[PP] encodent exactement un BNN en SAT, ce qui permet aussi de vérifier l'équivalence de deux réseaux [§5]. Sur un réseau dense de cinq blocs, avec un timeout de 300 s, la robustesse à $\epsilon = 3$ n'est certifiée que pour au plus 21 des 200 images de MNIST-back-image [Table 2]. Henzinger et al. (2020) vérifient bit à bit des réseaux en virgule fixe ; ils montrent que le problème est PSPACE-difficile avec une spécification bit-vector [Théorème 1], et annoncent un gain allant jusqu'à trois ordres de grandeur sur les outils existants [Abstract], sur un réseau dense de 52 650 paramètres. Leurs tableaux de résultats sont mal extraits ; la fiche ne garantit pas les chiffres cellule par cellule.

L'écart d'échelle est de plusieurs ordres de grandeur : le survey de Huang et al. (2018, §4.2, §4.3)[PP] situe les méthodes à bornes approchées à quelques milliers, au plus 10 000, neurones cachés, et le concept paper EASA juge les méthodes formelles limitées à des évaluations locales (EASA, 2024, §C.3.1.6). Prouver une propriété d'un YOLO complet n'est pas à portée. Deux usages

restent plausibles *(déduit)* : vérifier des sous-composants (une tête, un moniteur binaire), et vérifier l'équivalence entre le réseau quantifié de référence et ce que calcule le circuit, ce qu'aucun article du corpus ne fait.

8.4 Surveillance à l'exécution et incertitude

Les moniteurs du corpus sont presque tous évalués sur des classifieurs :

— la probabilité softmax maximale (MSP) (Hendrycks and Gimpel, 2017) sépare CIFAR-10 de SUN avec une AUROC de 95 [Table 2], mais tombe à 61 sur une tâche de NLP presque dans la distribution [Table 8] ;

— les patrons d'activation on/off stockés dans un BDD (Cheng et al., 2018)[PP] : sur GTSRB, à $\gamma = 3$, le moniteur se tait 95,42 % du temps et, quand il alerte, l'image est mal classée dans 54,54 % des cas [Table I] ; les BDD limitent pratiquement la couche surveillée à environ 200 neurones [§II] ;

— les boîtes d'abstraction sur les couches cachées (Henzinger et al., 2019) : une entrée d'entraînement n'est jamais rejetée [Prop. 1], la mémoire ne dépend pas de la taille du jeu [§4.3] et le test prend moins d'une milliseconde par entrée sur un portable [§5] ;

— SafeML (Aslansefat et al., 2020)[PP] compare des distributions empiriques sur un tampon d'échantillons ; sur un cas jouet, il estime une exactitude minimale de 0,772 là où la vraie vaut 0,508 [Table 1], c'est-à-dire qu'il se trompe dans le sens dangereux.

Trois travaux portent sur la détection d'objets. Gaussian YOLOv3 (Choi et al., 2019)[PP] prédit une variance par coordonnée de boîte pour 0,04 % de FLOPs en plus [§3.1] et réduit les faux positifs de 41,40 % sur KITTI au seuil 0,5 [Table 4] ; c'est une incertitude aléatoire de localisation, sans calibration ni OOD, mesurée sur GTX 1080 Ti. Feng et al. (2020) comparent les méthodes d'incertitude sur RetinaNet : la mAP ne mesure pas la qualité de l'incertitude, la modélisation directe de la variance est essentielle, et MC-Dropout ou les ensembles n'apportent pas assez pour leur coût (2 à 3 FPS pour 10 passes de MC-Dropout) [§IV-D, §V-A]. Sous décalage de domaine (Lyft), la mAP s'effondre vers 5 à 6 pour toutes les variantes [Table IV, corps possiblement décalé à l'extraction]. Enfin, Andéol et al. (2023)[PP] appliquent la prédiction conforme à YOLOv5m, DETR et DiffusionDet sur 3 414 images de signaux ferroviaires : la couverture empirique des boîtes va de 0,92 à 1,00 pour une cible de 0,9 [Table 4]. La garantie ne porte que sur les vrais positifs [§4.1] : une personne manquée n'est pas couverte, et l'hypothèse d'échangeabilité ne tient pas sur une longue durée [§6].

Aucun de ces moniteurs n'est implanté sur FPGA, ni évalué sur des personnes en milieu industriel. Plusieurs se réduisent à des comparaisons ou à des additions (seuil sur la MSP, test d'appartenance à une boîte, marge conforme), ce qui les rend a priori peu coûteux en logique *(déduit)* ; leur coût réel et leur effet sur un réseau INT8 restent à mesurer.

8.5 Logiciel prédictible et traçabilité de l'implémentation

ACETONE (De Albuquerque Silva et al., 2022) génère du C séquentiel, à allocation statique, depuis un réseau entraîné, pour la traçabilité et le calcul de WCET exigés par DO-178C [§1, §3]. L'écart numérique au modèle est de l'ordre de 10^{-6} en FP32 [§5], mais la préservation de la sémantique est testée, pas prouvée. Le WCET est calculé par OTAWA sur un lpc2138, non représentatif du Cortex-A15 mesuré [§4.2], et l'analyse échoue faute de mémoire pour AlexNet et la variante déroulée [§5]. Rien d'équivalent n'existe dans le corpus pour un flot HLS ou RTL *(déduit)*.

Côté FPGA, les accélérateurs YOLO du corpus ne traitent pas la sûreté. Sur KV260, Fata and Elmannai (2026) renvoient la tolérance aux fautes à un travail futur [§VI], et leur latence varie selon la vidéo [Fig. 14]. Sur ZCU104, Karki et al. (2026)[PP] détectent des personnes sur un jeu CCTV non publié ; la mAP@0.5 :0.95 passe de 0,75 sur CPU à 0,71 sur le FPGA en INT8 pour YOLOv11n, et de 0,64 à 0,34 pour YOLOv8n sur KITTI [Table II]. Un tel écart, imprévisible d'un modèle à l'autre,

est exactement ce que l'objectif IMP-06 demanderait de borner *(déduit)*. Le constat rejoint celui de la section 9 : la précision n'est presque jamais remesurée sur la cible.

8.6 Détection de personnes et robustesse

La revue de Halme et al. (2018) donne le cadre fonctionnel du cas d'application (arrêt de sécurité, surveillance de vitesse et de distance de séparation) et ses difficultés : occultations, sensibilité des caméras 2D à la lumière et à la poussière, latences de perception non traitées [§4]. Elle n'évalue aucun détecteur de personnes appris. CrowdHuman (Shao et al., 2018)[PP] mesure la difficulté de l'occultation : 22,64 personnes par image en moyenne [Table 2], et un FPN atteint un taux d'échec moyen (mMR) de 50,42 sur le corps entier contre 10,08 sur Caltech [Table 7]. Ce sont des images du web, sans scène industrielle ni EPI. Le survey de ré-identification (Ye et al., 2020)[PP] note une forte chute des performances entre jeux de données et sous attaque adverse [§2.4.2], sans aucune mesure embarquée. Pour la sécurité, Thys et al. (2019)[PP] montrent qu'un patch imprimé fait tomber le rappel de YOLOv2 sur Inria de 100 à 26,46 % [Table 1] ; le 100 % vient de ce que les vérités terrain sont produites par le même détecteur [§4]. L'effet de la quantification et de la compilation FPGA sur cette vulnérabilité n'est pas étudié.

8.7 Bilan

Table 4 – Familles de garanties dans le corpus : cible matérielle, modèle étudié et limite principale pour un YOLO certifiable sur FPGA.

Famille	Travaux	Cible	Modèle	Limite principale
Cadres et guides	EASA [a], AMLAS, DEEL, Ashmore	aucune	générique	matériel hors périmètre ou renvoyé à AMC 20-152A ; aviation ou automobile
Processus appliqué	ARP6983 [b]	Jetson AGX Xavier	YOLOv3-tiny	pas de FPGA ; aucune métrique de détection publiée
Injection de fautes, radiations	Li, Libano [c], Gambardella	simulateur ; Zynq, ZU9EG, ZCU104	classifieurs (19 poids à NiN)	aucun détecteur ; pas de lien FIT–SIL
Mitigation logicielle	FT-ClipAct, Rech [d]	simulation ; GPU surtout	classifieurs	transfert vers FPGA quantifié non fait
Vérification formelle	Reluplex, BNN [e], QNN [f]	CPU (solveurs)	denses, $\leq 52\,650$ paramètres	échelle ; pas d'équivalence avec le circuit
Moniteurs OOD	MSP, patrons, boîtes, SafeML	CPU, GPU	classifieurs	jamais sur détecteur ni sur FPGA
Incertitude, conforme	Gaussian YOLOv3, POD [g], conforme [h]	GPU de bureau	YOLOv3, RetinaNet, YOLOv5m	faux négatifs non couverts ; coût FPGA inconnu
Code prédictible	ACETONE	CPU ARM	MLP, CNN FP32	pas de HLS/RTL ni de quantification
Personnes, robustesse	Halme, CrowdHuman, Re-ID, patch [i]	aucune ou GPU	FPN, YOLOv2	ni scène industrielle ni cible embarquée

[a] EASA and Daedalean (2020), EASA and Daedalean (2024), EASA (2024) ; [b] Belcaid et al. (2024) ; [c] Libano et al. (2019), Libano et al. (2020) ; [d] Rech (2024) ; [e] Narodytska et al. (2017)[PP] ; [f] Henzinger et al. (2020) ; [g] Feng et al. (2020) ; [h] Andéol et al. (2023)[PP] ; [i] Thys et al. (2019)[PP].

Le tableau 4 montre des briques qui ne se rejoignent pas. Les cadres disent quoi prouver mais pas comment le faire sur un FPGA ; les méthodes de fiabilité, de vérification et de surveillance savent faire sur un petit classifieur, presque jamais sur un détecteur, et jamais sur un détecteur implanté sur FPGA. Pour un YOLO certifiable sur FPGA, il manquerait au minimum *(déduit)* :

— une correspondance entre les objectifs EASA ou AMLAS et les normes industrielles (IEC 61508, ISO 13849, ISO/IEC TR 5469) ;
— une métrique de criticité propre à la détection de personnes, reliée à un taux de défaillance par heure, puis à un SIL ou un PL ;
— une campagne d'injection de fautes et un durcissement sélectif sur un YOLO quantifié, BRAM comprises ;
— une preuve, ou au moins une borne mesurée, de l'écart entre le modèle de référence et le circuit déployé ;
— un moniteur d'exécution implanté sur la même puce, au coût et à la latence mesurés ;
— un jeu de données de personnes en zone industrielle, sur lequel évaluer le tout.

L'EASA ajoute une tension avec l'axe apprentissage continu de ce document : le modèle doit être gelé à l'approbation (EASA, 2024, MOC LM-02). Ces manques sont repris parmi les gaps de la section 10.

9 Analyse critique : ce que les chiffres publiés permettent de conclure

9.1 La puissance : rarement mesurée sur la carte, jamais sur la puce FPGA

Recompte de `benchmarks.csv` sur les 101 lignes mesurées sur FPGA :

Table 5 – Portée de la puissance dans les lignes FPGA de `benchmarks.csv`.

Portée	Lignes	Origine
board (carte au wattmètre)	18	FINN 8, FINN-R 8, Zhang 2015 1, ONLAD 1
chip	0	—
board-dynamic	0	—
estimated (outil)	8	Manjunath, Que 2024, ImageHD
unspecified	31	dont SATAY, Zhai, REQ-YOLO, Malviya
puissance non rapportée (`n/r`)	44	—

Les 17 lignes *chip* de la base viennent toutes de Jetson, du Raspberry Pi 5 ou d'un ASIC. Les 18 lignes *board* FPGA viennent de quatre articles, dont trois antérieurs à 2019. Plusieurs comparaisons publiées mettent face à face des portées différentes :

— une puissance dynamique de quelques mW contre des puissances totales (Coutinho et al., 2019, Table 16) ;
— la carte entière contre le CPU seul et le GPU seul (Tsukada et al., 2020, §6.4) ;
— le TDP d'un Xeon pris comme puissance mesurée (Zhang et al., 2015, §5.2) ;
— une estimation de l'outil contre des mesures par nvidia-smi (Arockiaraj et al., 2026, §VII-D).

Le survey de Shuvo et al. (2023) rappelle, dans sa partie sur les conditions de test, qu'un processeur peut afficher une très faible consommation en omettant les accès à la mémoire externe, et qu'il faut séparer résultats simulés et mesurés [§IX-I].

9.2 Des métriques qui flattent

— **Latence amortie** : un débit inverse présenté comme une latence (Que et al., 2024, §IV-E).
— **GOp/s par DSP** : cette métrique efface les écarts de précision (de W1A8 à W32A32) et de carte (Montgomerie-Corcoran et al., 2023, Table III). Venieris et al. la jugeaient valable seulement dans une même famille de FPGA (Venieris et al., 2018, §3.1).
— **GOPS/W dynamiques** face à des GOPS/W totaux (Zhai et al., 2023, Table 8).

— **GOPS projetés** sur un FPGA hypothétique, comparés à des GOPS mesurés (Coutinho et al., 2019, Tables 16–17).
— **GOp/s seul**, sans FPS, pour un détecteur (Blott et al., 2018, Table 5).
— **Précisions non comparables** : IoU d'un seul objet (DAC-SDC), AP à une classe (Zhai), « accuracy » sur 72 images (Al Amin), « accuracy » non définie d'un débruiteur (arxiv-2301.07050[PP]).

Shuvo et al. le formulent ainsi : « EI requires a universal and comprehensive set of quality metrics to execute meaningful comparisons of DL models, optimization algorithms, and hardware platforms » [§XIV-C]. La grille de Venieris et al. (débit, latence à batch 1, ressources, puissance, précision, mémoire) existe depuis 2018 ; aucune fiche du corpus ne la reprend en entier.

9.3 La précision n'est presque jamais remesurée sur la cible

Quantifier et compiler un réseau change sa précision, et la plupart des articles ne la remesurent pas sur la carte :
— AP identique en FP32 et en INT16 à trois décimales (Zhai et al., 2023, Table 8) ;
— quantification simulée par ONNX Runtime (Montgomerie-Corcoran et al., 2023, §IV-A) ;
— comparaison au C++ généré, pas au FPGA (Aarrestad et al., 2021, §8) ;
— AUC mesurées en TensorFlow (Tsukada et al., 2020, §5) ;
— seules des pertes relatives de mAP sont données (Ding et al., 2019, §6.2).

Quand la mesure sur cible est faite, elle révèle une perte :
— 85,02 % sur FPGA contre 88,8 % sur CPU (Malviya et al., 2022, §IV-B) ;
— 93,38 % contre 93,4 % (Coutinho et al., 2019, §V-D1).

Hors FPGA, la conversion peut être catastrophique : OpenVINO fait tomber YOLOv3-tiny à 0 % de confiance moyenne sur NCS2 (Feng et al., 2022, Table 3). La PTQ INT8 de TensorRT coûte de 6,4 à 12,3 points de mAP50-95 (Rey et al., 2025, Table 4).

9.4 Baselines GPU embarqué faibles ou absentes

Quand un article FPGA se compare à un GPU embarqué, la baseline est ancienne ou sous-optimisée :
— TX2 sans TensorRT mentionné (Ding et al., 2019, Table 2) ;
— TX2 de 2017, framework `n/r` (Montgomerie-Corcoran et al., 2023, Table IV) ;
— Jetson Nano reprise d'un autre article, avec un autre modèle (Zhai et al., 2023, Table 7) ;
— Jetson AGX Orin exécutée en PyTorch à batch 1 (Arockiaraj et al., 2026, §VII-C).

Les deux protocoles équitables du corpus sont anciens : fpgaConvNet face au Tegra X1 (2017), à budget de puissance égal, et DAC-SDC (2018), mesuré par les organisateurs. Aucune source du corpus ne compare une FPGA embarquée à une Jetson Orin.

9.5 Reproductibilité

Sur les 33 fiches expérimentales, aucune ne publie explicitement de bitstream. Sept publient ou annoncent un code propre : Magalhães, FINN-R en partie, la branche hls4ml de l'AE du LHC, REQ-YOLO (probablement la partie entraînement), DAC-SDC (code des meilleures équipes), le streamer de FCMP et l'extension HAWQ de Campos. Réserve : la lecture se limite au texte extrait, et les liens en note de bas de page sont souvent perdus.

10 Zones non couvertes et pistes de recherche

Les gaps ci-dessous viennent de `gaps.md` (8e passage, 01/10/2026), établi sur les 101 fiches de la base et les 299 lignes de `benchmarks.csv`, dont 216 sur FPGA. La carte compte 36 gaps,

dont 29 étayés. Un gap *étayé* est constaté dans plusieurs fiches ; cela ne prouve pas qu'il est absent de la littérature. Un gap à l'état d'*hypothèse* ne repose que sur l'absence de papiers dans la base. Le tableau 6 reprend les gaps d'architecture, de mesure et d'apprentissage ; le tableau 7 ceux de sûreté et de détection de personnes, deux axes ajoutés à la base avec le thème de thèse (détail en section 8).

Trois gaps du passage précédent ont changé, parce qu'une fiche nouvelle les contredit :

— « autoencodeur complet sur une petite FPGA » est supprimé. Un CAE complet, score par erreur de reconstruction, tourne en INT8 sur le DPU d'une ZCU104 (Castelino et al., 2024, §IV-B, Table VI), et une compression apprise place encodeur et décodeur sur ZCU102 (Mazouz et al., 2025, Table X)[PP]. Le coût des déconvolutions sur une puce plus petite passe dans le gap de l'anomalie visuelle ;

— « les apprentissages sur FPGA évitent la rétropropagation » devient « rétropropagation lente, en flottant, énergie non mesurée ». EF-Train fait FP / BP / WU en FP32 sur ZCU102, à 2,08 ms par image pour un petit CNN CIFAR-10 à batch 128 (Tang et al., 2022, Table 7)[PP] ; Chameleon le fait en FP16, à 413 ms et 1,22 J par image, contre 33 ms sur Jetson Nano (Aggarwal et al., 2024, Table VI) ;

— « CL sur FPGA : précision faible » devient « énergie de portée non précisée ». SLDA sur DPU atteint 78,79 % sur CORe50 (objet), à 9,00 W lus sur les rails sans portée déclarée (Piyasena et al., 2021, Table V).

Gap : la sûreté du détecteur embarqué. Les deux moitiés du sujet de thèse existent, mais séparément. Les études de fiabilité sur FPGA portent sur de petits classifieurs : 19 et 1 848 poids (Libano et al., 2019, §I, §III), un QNN FINN sur MNIST et CIFAR-10 (Gambardella et al., 2019, §V). Inversement, aucun des 18 papiers de l'axe `yolo-fpga` ne traite la sûreté, et le seul YOLO « certifié » du corpus tourne sur Jetson (Belcaid et al., 2024, §V). Les guides renvoient l'implémentation à plus tard : l'EASA écarte la conformité du matériel spécialisé (EASA and Daedalean, 2020, §4.2.3) et n'accepte pas l'apprentissage en ligne (EASA, 2024, MOC LM-02).

Gap : la détection de personnes sur FPGA. Un seul papier mesure un détecteur de personnes sur FPGA : YOLOv8, v11 et v26 nano en INT8 sur ZCU104, mAP@0.5 :0.95 de 0,66 à 0,79 sur un jeu maison non publié, 34,05 FPS de bout en bout et une puissance estimée par Vivado (Karki et al., 2026, Table II, §IV-C)[PP]. Aucun travail ne donne de taux de personnes manquées à taux de fausse alarme fixé.

10.1 Quatre questions de thèse candidates

Les pistes suivantes sont des propositions de ce document *(déduit)*. Elles combinent des gaps étayés et s'appuient sur des briques déjà présentes dans le corpus. Elles partagent un même objet expérimental : un YOLO quantifié de détection de personnes sur KV260.

1. **Un détecteur de personnes certifiable sur SoC FPGA.** Établir, pour un même YOLO, une chaîne de preuve d'équivalence entre le modèle QONNX et le bitstream, une borne de latence mesurée (pire cas, gigue) et un moniteur d'exécution sur la même puce. Briques existantes :
   — le YOLOv3-tiny allégé en w2a4 par FINN sur Zynq-7020 (Gutierrez et al., 2026)[PP], flot dataflow plus inspectable que la DPU *(déduit)* ;
   — la criticité par couche et le TMR sélectif de Libano et al. (2019) ;
   — la prédiction conforme sur YOLO (Andéol et al., 2023)[PP] et le moniteur à boîtes d'abstraction (Henzinger et al., 2019).
   Verrou : relier ces mesures à un taux de défaillance dangereuse par heure, dans un cadre de sécurité machine que le corpus ne contient pas encore.

TABLE 6 – Gaps FPGA d'architecture, de mesure et d'apprentissage : statut dans le corpus et piste principale.

Gap	Statut	Piste
Autoencodeurs		
Anomalie visuelle (images RGB) par AE sur FPGA embarquée	étayé (15 fiches) ; un contre-exemple partiel (hyperspectral, seuil réglé sur le test)	chaîne CAE INT8 de Vitis AI sur MVTec AD et KV260, seuil fixé hors du test, AUROC et puissance de carte ; coût des déconvolutions en dataflow
Compression apprise embarquée avec codage entropique	hypothèse (4 fiches)	chaîne complète sur plateforme à basse consommation, énergie par image mesurée
VAE en matériel pour un latent de grande dimension	hypothèse (2 fiches)	coût d'un générateur gaussien parallèle ; score D_{KL} déterministe en alternative
Détection		
Protocole de mesure commun pour YOLO sur FPGA	étayé (24 fiches)	mAP sur la cible, batch 1 de bout en bout avec pire cas et gigue, puissance de puce et de carte
Détection en dataflow quantifié sur SoC à mémoire limitée	étayé (13 fiches), nuancé	YOLO récent en W4–W8 par FINN ou SATAY sur KV260, face à la DPU sur la même carte
Petits objets : effet de la quantification par taille d'objet	étayé (2 fiches) pour l'absence ; hypothèse sinon	AP_{small} COCO mesurée sur la cible
Post-traitement et interaction PS–PL	étayé (14 fiches)	NMS matériel à côté de la DPU, latence de bout en bout et borne
Mesure et comparaison		
Précision non vérifiée sur la cible	étayé (20 fiches), nuancé	mAP COCO val sur la carte, étape par étape (quantification, compilation, opérateurs, NMS)
Baseline Jetson récente mesurée dans les mêmes conditions	étayé (27 fiches)	même modèle, même précision numérique, même batch, même méthode de puissance sur KV260 et Orin Nano
Puissance puce contre carte	étayé (40 fiches + `benchmarks.csv`)	télémétrie de la puce + wattmètre de carte + puissance au repos sur la même carte
Métriques flatteuses et comparaisons biaisées	étayé (20 fiches)	recalculer les rapports annoncés à métriques imposées
Code et bitstreams rarement publiés	étayé ($\approx$ 55 fiches expérimentales)	publier projet, bitstream et scripts de mesure
Toolflows		
Comparaison croisée hls4ml / FINN / Vitis AI	étayé (12 fiches), nuancé	un même YOLO à opérateurs ReLU exporté en QONNX, trois flots, une carte
Réseaux qui ne tiennent pas on-chip	étayé (13 fiches)	quantification agressive + FCMP ou éviction SMOF, coût à batch 1 et variabilité temporelle
Modèle de puissance validé dans les toolflows	hypothèse (outils) ; étayé (7 fiches) pour l'estimation non validée	vérifier la documentation actuelle de FINN et hls4ml
NAS et quantification mixte guidées par des mesures FPGA	hypothèse (nuancée)	HAWQ + ILP appliqués à la tête d'un YOLO
Apprentissage continu		
Rétropropagation sur SoC FPGA : lente, en flottant, énergie non mesurée	étayé (7 fiches)	queue de réseau en précision réduite en PL, étage figé sur la DPU
Adaptation à la dérive en détection d'anomalies	hypothèse (5 fiches)	ONLAD sur les caractéristiques d'un encodeur CNN figé
CL sur FPGA : énergie de portée non précisée ou estimée	étayé (6 fiches)	remesurer SLDA avec wattmètre et télémétrie
Apprentissage continu limité à la classification	étayé (13 fiches), nuancé (détection incrémentale sans FPGA)	Latent Distillation sur une tête YOLO, backbone figé sur la DPU

TABLE 7 – Gaps de sûreté, de certification et de détection de personnes : statut dans le corpus et piste principale. Aucun texte normatif n'est dans la base.

Gap	Statut	Piste
Sûreté, fiabilité et certification		
Aucun détecteur quantifié sur FPGA dans les travaux de fiabilité, de vérification ou de surveillance	étayé (25 fiches)	un YOLOv3-tiny ou v8n sur KV260, objet commun aux expériences de fiabilité, de mAP sur cible et de moniteur
Fiabilité SEU d'un détecteur : criticité par couche, TMR sélectif, DPU contre dataflow	étayé (9 fiches)	injection (configuration, BRAM, multi-fautes) sur un YOLO INT8 FINN et DPU ; durcissement sélectif chiffré en LUT et BRAM
Critère d'erreur dangereuse pour la détection de personnes ; FIT et précision vers SIL / PL	étayé (11 fiches)	non-détection définie sur une séquence ; taux de défaillance par heure combinant inexactitude et fautes
Preuve d'équivalence modèle → bitstream ; qualification des toolflows	étayé (14 fiches)	sorties bit à bit du modèle QONNX et du bitstream ; analyse TQL de FINN ou Vitis AI
Vérification formelle non scalable aux CNN de détection	étayé (4 fiches) ; hypothèse pour les outils récents	vérifier un moniteur ou un post-traitement RTL, plus petit que le réseau
Moniteurs d'exécution (OOD, incertitude, conforme) non portés sur FPGA	étayé (20 fiches)	prédiction conforme, puis moniteur à boîtes en PL ; surcoût mesuré sur KV260
Correspondance des cadres aéronautique et automobile avec les normes industrielles	étayé (11 fiches) ; hypothèse pour la littérature	tableau objectif par objectif vers IEC 61508, ISO 13849 et ISO/IEC TR 5469
Déterminisme du FPGA : WCET et gigue non mesurés	étayé (12 fiches)	traces de latence sur 10^5 images, DPU, FINN et TensorRT
Apprentissage en ligne exclu des cadres de certification	étayé (13 fiches)	SLDA StaticCov avec test de non-régression avant activation
Détection de personnes en milieu industriel		
Jeu de données et cas d'assurance en zone industrielle	étayé (8 fiches) ; hypothèse pour la littérature	recherche de jeux EPI ; ODD d'exemple selon AMLAS
Détecteurs de personnes quantifiés sur FPGA	étayé (9 fiches) ; un seul travail mesuré	YOLOv8n sur CrowdHuman, rappel à FPPI fixé sur KV260, flottant et INT8
Chaîne détection → suivi → ré-identification sur FPGA	étayé (4 fiches) ; hypothèse pour la Re-ID	suivi simple (SORT) sur le PS de la KV260
Robustesse adverse des détecteurs quantifiés	hypothèse (3 fiches)	rejouer un patch adverse sur le YOLO quantifié, avec et sans moniteur

2. **Un protocole de mesure et une comparaison croisée des flots.** Un même YOLO compilé par Vitis AI, et par FINN ou hls4ml via QONNX, sur KV260, face à une Jetson Orin Nano. Mesures : mAP COCO et rappel de personnes sur la cible, latence de bout en bout et sa distribution, puissance de puce et de carte. Le protocole partirait du cadre d'Schlosser et al. (2023) (mAP sur la carte, puissance de carte, repos déduit) et de ceux de DAC-SDC (Xu et al., 2021), de FINN (Umuroglu et al., 2016) et d'Alqahtani et al. (Alqahtani et al., 2024)[PP].
3. **Détection d'anomalies visuelles par autoencodeur convolutif sur SoC FPGA.** Briques existantes :
   — le CAE INT8 sur DPU et son score par reconstruction (Castelino et al., 2024), dont le seuil est réglé sur le test ;
   — l'encodeur ternaire paramétrable de Manjunath et al. (2021) ;
   — le score calculé sur la puce et le VAE réduit à l'encodeur de Govorkova et al. (2021).
   Verrou : une AUROC mesurée sur la carte, sur MVTec AD, avec un seuil fixé hors du test, sur KV260 ou en dataflow. Le même score pourrait servir de moniteur hors distribution pour le détecteur *(déduit)*.
4. **Adaptation en ligne certifiable par construction.** Un étage figé INT8 sur la DPU et une petite tête apprenante : SLDA (Piyasena et al., 2021), recette QLR-CL (Ravaglia et al., 2021) ou OS-ELM pour la dérive (Tsukada et al., 2020). Un test de non-régression sur un jeu de référence embarqué et un retour arrière précéderaient toute activation des nouveaux paramètres. Mesures : latence et énergie par mise à jour, surcoût de la validation. L'extension à la détection reste ouverte : la détection incrémentale n'existe que sans FPGA (Pasti et al., 2024)[PP].

11 Conclusion

Sur FPGA, la vision embarquée dispose d'une méthode de dimensionnement éprouvée (roofline, tuilage des boucles), d'outils ouverts pour les réseaux quantifiés (FINN, hls4ml) et, depuis SATAY, d'un toolflow streaming pour YOLO. Le corpus montre pourtant un écart persistant entre ce que les outils démontrent et ce qu'exige une application de vision sûre sur une petite carte :

— les démonstrations portent sur de petits réseaux ou de grosses puces ;
— les réseaux profonds butent sur la mémoire on-chip ;
— les mesures de puissance et de précision sur la cible sont rares, et aucune ne donne de pire cas de latence ;
— l'anomalie visuelle par autoencodeur n'est traitée que sur un cas hyperspectral, et aucun article n'adapte en ligne un détecteur ;
— aucun détecteur sur FPGA n'a été soumis à une injection de fautes, à une vérification ou à un moniteur d'exécution, et aucun cadre du corpus ne relie ces garanties aux normes de sécurité industrielles (section 8).

Ces manques sont autant de sujets possibles : ils combinent une contribution d'architecture (tenir sur KV260), une contribution méthodologique (mesurer correctement) et une contribution de sûreté (prouver que le circuit déployé fait ce que fait le modèle).

Pistes concrètes.

1. **Expérience à reproduire** : le cadre de Schlosser et al. (2023, §4.1, Table 1) (mAP COCO mesurée sur la carte, puissance de carte, repos déduit), transposé à un YOLOv8n sur KV260 par Vitis AI et par FINN, face à une Jetson Orin Nano. Il faudrait y ajouter la puissance de puce et une distribution de latence (p99, max) sur un grand nombre d'images.
2. **Sûreté** : sur le YOLOv3-tiny w2a4 de Gutierrez et al. (2026)[PP], comparer bit à bit les sorties du modèle QONNX et du bitstream, puis appliquer la méthode d'injection et de criticité par couche de Libano et al. (2019), BRAM comprises.

3. **Gap à creuser** : un autoencodeur convolutif d'anomalie visuelle sur MVTec AD et KV260, en reprenant la chaîne de Castelino et al. (2024) avec un seuil fixé hors du test ; le même score servirait de moniteur hors distribution pour le détecteur *(déduit)*.
4. **Lectures à ajouter** : ISO/IEC TR 5469 (ou une présentation publique), des travaux de sécurité machine avec apprentissage, et des jeux de détection de personnes avec EPI ; aucun n'est dans la base.

Références

Thea Aarrestad, Vladimir Loncar, Nicolò Ghielmetti, Maurizio Pierini, Sioni Summers, Jennifer Ngadiuba, Christoffer Petersson, Hampus Linander, Yutaro Iiyama, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Dylan Rankin, Sergo Jindariani, Kevin Pedro, Nhan Tran, Mia Liu, Edward Kreinar, Zhenbin Wu, and Duc Hoang. Fast convolutional neural networks on FPGAs with hls4ml. *Mach. Learn. : Sci. Technol. 2 045015 (2021)*, 2021. doi : 10.1088/2632-2153/ac0ea1. URL `https://arxiv.org/abs/2101.05108`. arXiv :2101.05108.

Kamel Abdelouahab, Maxime Pelcat, Jocelyn Serot, and François Berry. Accelerating CNN inference on FPGAs : A Survey. Technical Report, Université Clermont Auvergne (HAL), 2018, 2018. URL `https://arxiv.org/abs/1806.01683`. arXiv :1806.01683. Preprint non relu par les pairs.

Shivam Aggarwal, Kuluhan Binici, and Tulika Mitra. Chameleon : Dual Memory Replay for Online Continual Learning on Edge Devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 43(6)*, 2024. URL `https://ieeexplore.ieee.org/document/10375748/`.

Rashed Al Amin and Roman Obermaisser. Real-Time Object Detection and Classification using YOLO for Edge FPGAs. In *2025 International Symposium ELMAR*, 2025. doi : 10.1109/ELMAR66948.2025.11193980. URL `https://arxiv.org/abs/2507.18174`. arXiv :2507.18174.

Daghash K Alqahtani, Muhammad Aamir Cheema, Maria A Rodriguez, and Adel N Toosi. A Comprehensive Evaluation of Deep Learning Object Detection Models on Heterogeneous Edge Devices. arXiv preprint, 2024. URL `https://arxiv.org/abs/2409.16808`. arXiv :2409.16808. Preprint non relu par les pairs.

Léo Andéol, Thomas Fel, Florence De Grancey, and Luca Mossina. Confident Object Detection via Conformal Prediction and Conformal Risk Control : an Application to Railway Signaling. arXiv preprint, 2023. URL `https://arxiv.org/abs/2304.06052`. arXiv :2304.06052. Preprint non relu par les pairs.

Jebacyril Arockiaraj, Dhruv Parikh, and Viktor Prasanna. ImageHD : Energy-Efficient On-Device Continual Learning of Visual Representations via Hyperdimensional Computing. In *2026 IEEE 34th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2026. doi : 10.1109/FCCM68464.2026.00041. URL `https://arxiv.org/abs/2604.21280`. arXiv :2604.21280.

Rob Ashmore, Radu Calinescu, and Colin Paterson. Assuring the Machine Learning Lifecycle : Desiderata, Methods, and Challenges. arXiv preprint, 2019. URL `https://arxiv.org/abs/1905.04223`. arXiv :1905.04223. Preprint non relu par les pairs.

Koorosh Aslansefat, Ioannis Sorokos, Declan Whiting, Ramin Tavakoli Kolagari, and Yiannis Papadopoulos. SafeML : Safety Monitoring of Machine Learning Classifiers through Statistical Difference Measure. arXiv preprint, 2020. URL `https://arxiv.org/abs/2005.13166`. arXiv :2005.13166. Preprint non relu par les pairs.

Mohammed Belcaid, Eric Bonnafous, Louis Crison, Christophe Faure, Eric Jenn, and Claire Pagetti. Certified ML Object Detection for Surveillance Missions. In *ERTS 2024 (12th European Congress*

on Embedded Real Time Software and Systems), Toulouse, 2024. URL https://arxiv.org/abs/2406.12362.

Michaela Blott, Thomas B. Preußer, Nicholas J. Fraser, Giulio Gambardella, Kenneth O'brien, Yaman Umuroglu, Miriam Leeser, and Kees Vissers. FINN-R : An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks. *ACM Transactions on Reconfigurable Technology and Systems 11(3)*, 2018. doi : 10.1145/3242897. URL https://arxiv.org/abs/1809.04570. arXiv :1809.04570.

Javier Campos, Jovan Mitrevski, Nhan Tran, Zhen Dong, Amir Gholaminejad, Michael W. Mahoney, and Javier Duarte. End-to-end codesign of Hessian-aware quantized neural networks for FPGAs. *ACM Trans. Reconfigurable Technol. Syst. 17(3), 36*, 2024. doi : 10.1145/3662000. URL https://doi.org/10.1145/3662000.

Cornell Castelino, Shashwat Khandelwal, Shanker Shreejith, and Sharatchandra Varma Bogaraju. An Energy-Efficient Artefact Detection Accelerator on FPGAs for Hyper-Spectral Satellite Imagery. In *27th Euromicro Conference on Digital System Design (DSD), 2024*, 2024. URL https://arxiv.org/abs/2407.17647.

Chih-Hong Cheng, Georg Nührenberg, and Hirotoshi Yasuoka. Runtime Monitoring Neuron Activation Patterns. arXiv preprint, 2018. URL https://arxiv.org/abs/1809.06573. arXiv :1809.06573. Preprint non relu par les pairs.

Jiwoong Choi, Dayoung Chun, Hyun Kim, and Hyuk-Jae Lee. Gaussian YOLOv3 : An Accurate and Fast Object Detector Using Localization Uncertainty for Autonomous Driving. arXiv preprint, 2019. URL https://arxiv.org/abs/1904.04620. arXiv :1904.04620. Preprint non relu par les pairs.

Maria G. F. Coutinho, Matheus F. Torquato, and Marcelo A. C. Fernandes. Deep Neural Network Hardware Implementation Based on Stacked Sparse Autoencoder. *IEEE Access 7, 40674-40694*, 2019. doi : 10.1109/ACCESS.2019.2907261. URL https://doi.org/10.1109/ACCESS.2019.2907261.

Yajie Cui, Zhaoxiang Liu, and Shiguo Lian. A Survey on Unsupervised Anomaly Detection Algorithms for Industrial Images. *IEEE Access*, 2023. doi : 10.1109/ACCESS.2023.3282993. URL https://doi.org/10.1109/ACCESS.2023.3282993.

Iryna De Albuquerque Silva, Thomas Carle, Adrien Gauffriau, and Claire Pagetti. ACETONE : Predictable Programming Framework for ML Applications in Safety-Critical Systems. In *34th Euromicro Conference on Real-Time Systems (ECRTS 2022), LIPIcs 231*, 2022. doi : 10.4230/LIPIcs.ECRTS.2022.3. URL https://doi.org/10.4230/LIPIcs.ECRTS.2022.3.

Hervé Delseny, Christophe Gabreau, Adrien Gauffriau, Bernard Beaudouin, Ludovic Ponsolle, Lucian Alecu, Hugues Bonnin, Brice Beltran, Didier Duchel, Jean-Brice Ginestet, Alexandre Hervieu, Ghilaine Martinez, Sylvain Pasquet, Kevin Delmas, Claire Pagetti, Jean-Marc Gabriel, Camille Chapdelaine, Sylvaine Picard, Mathieu Damour, Cyril Cappi, Laurent Gardès, Florence De Grancey, Eric Jenn, Baptiste Lefevre, Gregory Flandin, Sébastien Gerchinovitz, Franck Mamalet, and Alexandre Albore. White Paper Machine Learning in Certified Systems. arXiv preprint, 2021. URL https://arxiv.org/abs/2103.10529. arXiv :2103.10529. Preprint non relu par les pairs.

Caiwen Ding, Shuo Wang, Ning Liu, Kaidi Xu, Yanzhi Wang, and Yun Liang. REQ-YOLO : A Resource-Aware, Efficient Quantization Framework for Object Detection on FPGAs. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays 33–42*, 2019. doi : 10.1145/3289602.3293904. URL https://doi.org/10.1145/3289602.3293904.

EASA. Concept Paper : Guidance for Level 1 & 2 Machine Learning Applications. Technical report, European Union Aviation Safety Agency (EASA), Concept Paper, Issue 02, 2024.

EASA and Daedalean. Concepts of Design Assurance for Neural Networks (CoDANN). Technical report, EASA AI Task Force and Daedalean AG, public report extract, version 1.0, 2020.

EASA and Daedalean. Concepts of Design Assurance for Neural Networks (CoDANN) II. Technical report, EASA AI Task Force and Daedalean AG, public extract, version 1.1, 2024.

Farah Fahim, Benjamin Hawks, Christian Herwig, James Hirschauer, Sergo Jindariani, Nhan Tran, Luca P Carloni, Giuseppe Di Guglielmo, Philip Harris, Jeffrey Krupa, Dylan Rankin, Manuel Blanco Valentin, Josiah Hester, Yingyi Luo, John Mamish, Seda Orgrenci-Memik, Thea Aarrestad, Hamza Javed, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, Sioni Summers, Javier Duarte, Scott Hauck, Shih-Chieh Hsu, Jennifer Ngadiuba, Mia Liu, Duc Hoang, Edward Kreinar, and Zhenbin Wu. hls4ml : An Open-Source Codesign Workflow to Empower Scientific Low-Power Machine Learning Devices. In *TinyML Research Symposium 2021*, 2021. URL `https://arxiv.org/abs/2103.05579`. arXiv :2103.05579.

John S. Fata and Wafa M. Elmannai. Low-Cost FPGA-Enhanced CNN Accelerator for Real-Time YOLO Object Detection and Classification. *IEEE Access 14, 34614-34642*, 2026. doi : 10.1109/ACCESS.2026.3669451. URL `https://doi.org/10.1109/ACCESS.2026.3669451`.

Di Feng, Ali Harakeh, Steven Waslander, and Klaus Dietmayer. A Review and Comparative Study on Probabilistic Object Detection in Autonomous Driving. *IEEE Transactions on Intelligent Transportation Systems*, 2020. doi : 10.1109/TITS.2021.3096854. URL `https://arxiv.org/abs/2011.10671`.

Haogang Feng, Gaoze Mu, Shida Zhong, Peichang Zhang, and Tao Yuan. Benchmark Analysis of YOLO Performance on Edge Intelligence Devices. *Cryptography 6(2), 16*, 2022. doi : 10.3390/cryptography6020016. URL `https://doi.org/10.3390/cryptography6020016`.

Giulio Gambardella, Johannes Kappauf, Michaela Blott, Christoph Doehring, Martin Kumm, Peter Zipf, and Kees Vissers. Efficient Error-Tolerant Quantized Neural Network Accelerators. In *2019 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2019. doi : 10.1109/DFT.2019.8875314. URL `https://arxiv.org/abs/1912.07394`.

Ekaterina Govorkova, Ema Puljak, Thea Aarrestad, Thomas James, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, Nicolò Ghielmetti, Maksymilian Graczyk, Sioni Summers, Jennifer Ngadiuba, Thong Q Nguyen, Javier Duarte, and Zhenbin Wu. Autoencoders on FPGAs for real-time, unsupervised new physics detection at 40 MHz at the Large Hadron Collider. *Nature Machine Intelligence 4, 154 (2022)*, 2021. doi : 10.1038/s42256-022-00441-3. URL `https://arxiv.org/abs/2108.03986`. arXiv :2108.03986.

Katya Govorkova, Julian Garcia Pardinas, Vladimir Loncar, Victoria Nguyen, Sebastian Schmitt, Marco Pizzichemi, Loris Martinazzoli, and Eluned Anne Smith. Enabling Low-Latency Machine learning on Radiation-Hard FPGAs with hls4ml. *Machine Learning : Science and Technology*, 2026. doi : 10.1088/2632-2153/ae8218. URL `https://arxiv.org/abs/2602.15751`. arXiv :2602.15751.

Giorgia Guerrisi, Fabio Del Frate, and Giovanni Schiavon. Artificial Intelligence Based On-Board Image Compression for the Φ-Sat-2 Mission. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing 16, 8063-8075*, 2023. doi : 10.1109/JSTARS.2023.3296485. URL `https://doi.org/10.1109/JSTARS.2023.3296485`.

Daniel Gutierrez, Antonio Cuesta, Jorge Fe, Bruno Gutierrez, and Rashed Al Koutayni. NPU Accelerator : Quantized Real-Time Vehicle Detection on PYNQ-Z1 Using FINN. arXiv preprint,

2026. URL https://arxiv.org/abs/2609.24757. arXiv :2609.24757. Preprint non relu par les pairs.

Roni-Jussi Halme, Minna Lanz, Joni Kämäräinen, Roel Pieters, Jyrki Latokartano, and Antti Hietanen. Review of vision-based safety systems for human-robot collaboration. *Procedia CIRP 72, 111-116*, 2018. doi : 10.1016/j.procir.2018.03.043. URL https://doi.org/10.1016/j.procir.2018.03.043.

Richard Hawkins, Colin Paterson, Chiara Picardi, Yan Jia, Radu Calinescu, and Ibrahim Habli. Guidance on the Assurance of Machine Learning in Autonomous Systems (AMLAS). arXiv preprint, 2021. URL https://arxiv.org/abs/2102.01564. arXiv :2102.01564. Preprint non relu par les pairs.

Dan Hendrycks and Kevin Gimpel. A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks. In *International Conference on Learning Representations 2017*, 2017. URL https://arxiv.org/abs/1610.02136.

Thomas A Henzinger, Anna Lukina, and Christian Schilling. Outside the Box : Abstraction-Based Monitoring of Neural Networks. In *ECAI 2020*, 2019. doi : 10.3233/FAIA200375. URL https://arxiv.org/abs/1911.09032.

Thomas A Henzinger, Mathias Lechner, and Đorđe Žikelić. Scalable Verification of Quantized Neural Networks (Technical Report). In *AAAI 2021*, 2020. URL https://arxiv.org/abs/2012.08185.

Le-Ha Hoang, Muhammad Abdullah Hanif, and Muhammad Shafique. FT-ClipAct : Resilience Analysis of Deep Neural Networks and Improving their Fault Tolerance using Clipped Activation. arXiv preprint, 2019. URL https://arxiv.org/abs/1912.00941. arXiv :1912.00941. Preprint non relu par les pairs.

Seyed Hani Hozhabr and Roberto Giorgi. A Survey on Real-Time Object Detection on FPGAs. *IEEE Access 13, 38195-38238*, 2025. doi : 10.1109/ACCESS.2025.3544515. URL https://doi.org/10.1109/ACCESS.2025.3544515.

Xiaowei Huang, Daniel Kroening, Wenjie Ruan, James Sharp, Youcheng Sun, Emese Thamo, Min Wu, and Xinping Yi. A Survey of Safety and Trustworthiness of Deep Neural Networks : Verification, Testing, Adversarial Attack and Defence, and Interpretability. arXiv preprint, 2018. URL https://arxiv.org/abs/1812.08342. arXiv :1812.08342. Preprint non relu par les pairs.

Murat Isik, Matthew Oldland, and Lifeng Zhou. An Energy-Efficient Reconfigurable Autoencoder Implementation on FPGA. arXiv preprint, 2023. URL https://arxiv.org/abs/2301.07050. arXiv :2301.07050. Preprint non relu par les pairs.

Suraj Karki, Qazi Arbab Ahmed, and Thorsten Jungeblut. No Attention, No Problem : DPU-Aware Attention Approximation in Modern YOLO on FPGA. arXiv preprint, 2026. URL https://arxiv.org/abs/2607.13106. arXiv :2607.13106. Preprint non relu par les pairs.

Guy Katz, Clark Barrett, David Dill, Kyle Julian, and Mykel Kochenderfer. Reluplex : An Efficient SMT Solver for Verifying Deep Neural Networks. arXiv preprint, 2017. URL https://arxiv.org/abs/1702.01135. arXiv :1702.01135. Preprint non relu par les pairs.

Guanpeng Li, Siva Kumar Sastry Hari, Michael Sullivan, Timothy Tsai, Karthik Pattabiraman, Joel Emer, and Stephen W. Keckler. Understanding error propagation in deep learning neural network (DNN) accelerators and applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis 8*, 2017. doi : 10.1145/3126908.3126964. URL https://doi.org/10.1145/3126908.3126964.

F. Libano, B. Wilson, J. Anderson, M. J. Wirthlin, C. Cazzaniga, C. Frost, and P. Rech. Selective Hardening for Neural Networks in FPGAs. *IEEE Transactions on Nuclear Science 66(1), 216-222*, 2019. doi : 10.1109/TNS.2018.2884460. URL https://doi.org/10.1109/TNS.2018.2884460.

F. Libano, B. Wilson, M. Wirthlin, P. Rech, and J. Brunhaver. Understanding the Impact of Quantization, Accuracy, and Radiation on the Reliability of Convolutional Neural Networks on FPGAs. *IEEE Transactions on Nuclear Science 67(7), 1478-1484*, 2020. doi : 10.1109/TNS.2020.2983662. URL https://doi.org/10.1109/TNS.2020.2983662.

Ji Lin, Wei-Ming Chen, Han Cai, Chuang Gan, and Song Han. MCUNetV2 : Memory-Efficient Patch-based Inference for Tiny Deep Learning. In *NeurIPS 2021 (d'après 2025-el-zeinaty-designing-object-detection-models-tinyml, Table 8 ; non imprimé dans la version arXiv)*, 2021. URL https://arxiv.org/abs/2110.15352. arXiv :2110.15352.

Sandro Costa Magalhães, Filipe Neves Santos, Pedro Machado, António Paulo Moreira, and Jorge Dias. Benchmarking Edge Computing Devices for Grape Bunches and Trunks Detection using Accelerated Object Detection Single Shot MultiBox Deep Learning Models. *Engineering Applications of Artificial Intelligence 117, 105604 (2022)*, 2022. doi : 10.1016/j.engappai.2022.105604. URL https://arxiv.org/abs/2211.11647. arXiv :2211.11647.

Vaibhav Malviya, Indrani Mukherjee, and Siddharth Tallur. Edge-Compatible Convolutional Autoencoder Implemented on FPGA for Anomaly Detection in Vibration Condition-Based Monitoring. *IEEE Sensors Letters 6(4), 1-4*, 2022. doi : 10.1109/LSENS.2022.3159972. URL https://doi.org/10.1109/LSENS.2022.3159972.

Nitheesh Kumar Manjunath, Aidin Shiri, Morteza Hosseini, Bharat Prakash, Nicholas R. Waytowich, and Tinoosh Mohsenin. An Energy Efficient EdgeAI Autoencoder Accelerator for Reinforcement Learning. *IEEE Open Journal of Circuits and Systems 2, 182-195*, 2021. doi : 10.1109/OJCAS.2020.3043737. URL https://doi.org/10.1109/OJCAS.2020.3043737.

Alaa Mazouz, Sumanta Chaudhuri, Marco Cagnanzzo, Mihai Mitrea, Enzo Tartaglione, and Attilio Fiandrotti. Lightweight Embedded FPGA Deployment of Learned Image Compression with Knowledge Distillation and Hybrid Quantization. arXiv preprint, 2025. URL https://arxiv.org/abs/2503.04832. arXiv :2503.04832. Preprint non relu par les pairs.

Alexander Montgomerie-Corcoran, Petros Toupas, Zhewen Yu, and Christos-Savvas Bouganis. SATAY : A Streaming Architecture Toolflow for Accelerating YOLO Models on FPGA Devices. In *2023 International Conference on Field Programmable Technology (ICFPT) 179-187*, 2023. doi : 10.1109/ICFPT59805.2023.00025. URL https://doi.org/10.1109/ICFPT59805.2023.00025.

Nina Narodytska, Shiva Prasad Kasiviswanathan, Leonid Ryzhyk, Mooly Sagiv, and Toby Walsh. Verifying Properties of Binarized Deep Neural Networks. arXiv preprint, 2017. URL https://arxiv.org/abs/1709.06662. arXiv :1709.06662. Preprint non relu par les pairs.

Francesco Pasti, Marina Ceccon, Davide Dalle Pezze, Francesco Paissan, Elisabetta Farella, Gian Antonio Susto, and Nicola Bellotto. Latent Distillation for Continual Object Detection at the Edge. arXiv preprint, 2024. URL https://arxiv.org/abs/2409.01872. arXiv :2409.01872. Preprint non relu par les pairs.

Lucian Petrica, Tobias Alonso, Mairin Kroes, Nicholas Fraser, Sorin Cotofana, and Michaela Blott. Memory-Efficient Dataflow Inference for Deep CNNs on FPGA. arXiv preprint, 2020. URL https://arxiv.org/abs/2011.07317. arXiv :2011.07317. Preprint non relu par les pairs.

Duvindu Piyasena, Siew-Kei Lam, and Meiqing Wu. Accelerating Continual Learning on Edge FPGA. In *2021 31st International Conference on Field-Programmable Logic and Applications (FPL) 294-300*, 2021. doi : 10.1109/FPL53798.2021.00059. URL https://doi.org/10.1109/FPL53798.2021.00059.

Zhiqiang Que, Minghao Zhang, Hongxiang Fan, He Li, Ce Guo, and Wayne Luk. Low Latency Variational Autoencoder on FPGAs. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems 14(2), 323-333*, 2024. doi : 10.1109/JETCAS.2024.3389660. URL https://doi.org/10.1109/JETCAS.2024.3389660.

Leonardo Ravaglia, Manuele Rusci, Davide Nadalini, Alessandro Capotondi, Francesco Conti, and Luca Benini. A TinyML Platform for On-Device Continual Learning with Quantized Latent Replays. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems 11.4 (2021), 789-802*, 2021. doi : 10.1109/JETCAS.2021.3121554. URL https://arxiv.org/abs/2110.10486. arXiv :2110.10486.

Paolo Rech. Artificial Neural Networks for Space and Safety-Critical Applications : Reliability Issues and Potential Solutions. *IEEE Transactions on Nuclear Science 71(4), 377-404*, 2024. doi : 10.1109/TNS.2024.3349956. URL https://doi.org/10.1109/TNS.2024.3349956.

Lucas Rey, Ana M Bernardos, Andrzej D Dobrzycki, David Carramiñana, Luca Bergesio, Juan A Besada, and José Ramón Casar. A Performance Analysis of You Only Look Once Models for Deployment on Constrained Computational Edge Devices in Drone Applications. *Electronics 2025, 14(3), 638*, 2025. doi : 10.3390/electronics14030638. URL https://arxiv.org/abs/2502.15737. arXiv :2502.15737.

Annachiara Ruospo, Ernesto Sanchez, Lucas Matana Luza, Luigi Dilillo, Marcello Traiola, and Alberto Bosio. A Survey on Deep Learning Resilience Assessment Methodologies. *Computer 56(2), 57-66*, 2023. doi : 10.1109/MC.2022.3217841. URL https://doi.org/10.1109/MC.2022.3217841.

Rick Salay, Rodrigo Queiroz, and Krzysztof Czarnecki. An Analysis of ISO 26262 : Using Machine Learning Safely in Automotive Software. arXiv preprint, 2017. URL https://arxiv.org/abs/1709.02435. arXiv :1709.02435. Preprint non relu par les pairs.

Safa Mohammed Sali, Mahmoud Meribout, and Ashiyana Abdul Majeed. Real Time FPGA Based CNNs for Detection, Classification, and Tracking in Autonomous Systems : State of the Art Designs and Optimizations. arXiv preprint, 2025. URL https://arxiv.org/abs/2509.04153. arXiv :2509.04153. Preprint non relu par les pairs.

Michael Schlosser, Daniel König, and Michael Teutsch. A Framework for Benchmarking Real-Time Embedded Object Detection. In *In : Andres, B., Bernard, F., Cremers, D., Frintrop, S., Goldlücke, B., Ihrke, I. (eds) Pattern Recognition. DAGM GCPR 2022. Lecture Notes in Computer Science, vol 13485. Springer, Cham*, 2023. doi : 10.1007/978-3-031-16788-1_33. URL https://arxiv.org/abs/2304.11580.

Jan-Frederik Schulte, Benjamin Ramhorst, Chang Sun, Jovan Mitrevski, Nicolò Ghielmetti, Enrico Lupi, Dimitrios Danopoulos, Vladimir Loncar, Javier Duarte, David Burnette, Lauri Laatu, Stylianos Tzelepis, Konstantinos Axiotis, Quentin Berthet, Haoyan Wang, Paul White, Suleyman Demirsoy, Marco Colombo, Thea Aarrestad, Sioni Summers, Maurizio Pierini, Giuseppe Di Guglielmo, Jennifer Ngadiuba, Javier Campos, Ben Hawks, Abhijith Gandrakota, Farah Fahim, Nhan Tran, George Constantinides, Zhiqiang Que, Wayne Luk, Alexander Tapper, Duc Hoang, Noah Paladino, Philip Harris, Bo-Cheng Lai, Manuel Valentin, Ryan Forelli, Seda Ogrenci, Lino Gerlach, Rian Flynn, Mia Liu, Daniel Diaz, Elham Khoda, Melissa Quinnan, Russell Solares, Santosh Parajuli, Mark Neubauer, Christian Herwig, Ho Fung Tsoi, Dylan Rankin, Shih-Chieh Hsu, and Scott Hauck. hls4ml : A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware. arXiv preprint, 2025. URL https://arxiv.org/abs/2512.01463. arXiv :2512.01463. Preprint non relu par les pairs.

Gesina Schwalbe and Martin Schels. A Survey on Methods for the Safety Assurance of Machine Learning Based Systems. In *10th European Congress on Embedded Real Time Software and Systems (ERTS 2020)*, 2020.

Shuai Shao, Zijian Zhao, Boxun Li, Tete Xiao, Gang Yu, Xiangyu Zhang, and Jian Sun. CrowdHuman : A Benchmark for Detecting Human in a Crowd. arXiv preprint, 2018. URL https://arxiv.org/abs/1805.00123. arXiv :1805.00123. Preprint non relu par les pairs.

Md. Maruf Hossain Shuvo, Syed Kamrul Islam, Jianlin Cheng, and Bashir I. Morshed. Efficient Acceleration of Deep Learning Inference on Resource-Constrained Edge Devices : A Review. *Proceedings of the IEEE 111(1), 42-91*, 2023. doi : 10.1109/JPROC.2022.3226481. URL https://doi.org/10.1109/JPROC.2022.3226481.

Heming Sun, Qingyang Yi, Fangzheng Lin, Lu Yu, Jiro Katto, and Masahiro Fujita. F-LIC : FPGA-based Learned Image Compression with a Fine-grained Pipeline. In *2022 IEEE Asian Solid-State Circuits Conference (A-SSCC) 1-3*, 2022. doi : 10.1109/A-SSCC56115.2022.9980666. URL https://doi.org/10.1109/A-SSCC56115.2022.9980666.

Yue Tang, Xinyi Zhang, Peipei Zhou, and Jingtong Hu. EF-Train : Enable Efficient On-device CNN Training on FPGA Through Data Reshaping for Online Adaptation or Personalization. arXiv preprint, 2022. URL https://arxiv.org/abs/2202.10935. arXiv :2202.10935. Preprint non relu par les pairs.

Simen Thys, Wiebe Van Ranst, and Toon Goedemé. Fooling automated surveillance cameras : adversarial patches to attack person detection. arXiv preprint, 2019. URL https://arxiv.org/abs/1904.08653. arXiv :1904.08653. Preprint non relu par les pairs.

Mineto Tsukada, Masaaki Kondo, and Hiroki Matsutani. A Neural Network-Based On-device Learning Anomaly Detector for Edge Devices. *IEEE Transactions on Computers*, 2020. doi : 10.1109/TC.2020.2973631. URL https://doi.org/10.1109/TC.2020.2973631.

Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. FINN : A Framework for Fast, Scalable Binarized Neural Network Inference. In *FPGA '17 : ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2016. doi : 10.1145/3020078.3021744. URL https://arxiv.org/abs/1612.07119. arXiv :1612.07119.

Stylianos I Venieris and Christos-Savvas Bouganis. fpgaConvNet : A Toolflow for Mapping Diverse Convolutional Neural Networks on Embedded FPGAs. In *NIPS 2017 Workshop on Machine Learning on the Phone and other Consumer Devices*, 2017. URL https://arxiv.org/abs/1711.08740. arXiv :1711.08740.

Stylianos I. Venieris, Alexandros Kouris, and Christos-Savvas Bouganis. Toolflows for Mapping Convolutional Neural Networks on FPGAs : A Survey and Future Directions. *ACM Comput. Surv. 51(3), 56*, 2018. doi : 10.1145/3186332. URL https://doi.org/10.1145/3186332.

Yoga Esa Wibowo, Cristian Cioflan, Thorir Mar Ingolfsson, Michael Hersche, Leo Zhao, Abbas Rahimi, and Luca Benini. 12 mJ per Class On-Device Online Few-Shot Class-Incremental Learning. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024. doi : 10.23919/DATE58400.2024.10546598. URL https://arxiv.org/abs/2403.07851. arXiv :2403.07851.

Xiaowei Xu, Xinyi Zhang, Bei Yu, Xiaobo Sharon Hu, Christopher Rowen, Jingtong Hu, and Yiyu Shi. DAC-SDC Low Power Object Detection Challenge for UAV Applications. *IEEE Transactions on Pattern Analysis and Machine Intelligence 43(2), 392-403*, 2021. doi : 10.1109/TPAMI.2019.2932429. URL https://doi.org/10.1109/TPAMI.2019.2932429.

Mang Ye, Jianbing Shen, Gaojie Lin, Tao Xiang, Ling Shao, and Steven C H Hoi. Deep Learning for Person Re-identification : A Survey and Outlook. arXiv preprint, 2020. URL https://arxiv.org/abs/2001.04193. arXiv :2001.04193. Preprint non relu par les pairs.

Jiaqi Zhai, Bin Li, Shunsen Lv, and Qinglei Zhou. FPGA-Based Vehicle Detection and Tracking Accelerator. *Sensors 23(4), 2208*, 2023. doi : 10.3390/s23042208. URL `https://doi.org/10.3390/s23042208`.

Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks. In *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays 161–170*, 2015. doi : 10.1145/2684746.2689060. URL `https://doi.org/10.1145/2684746.2689060`.